Good morning, and thank you for giving me this opportunity.

My name is Shaik Azeedh. I completed my Master of Computer Applications (MCA). I have around 5 years of experience as an AWS Cloud Administrator, where I have been responsible for managing and supporting cloud infrastructure in production environments.

In my current role, I work extensively with AWS services such as EC2, S3, IAM, VPC, EBS, Route 53, CloudWatch, Auto Scaling, and Elastic Load Balancer. My responsibilities include provisioning and managing EC2 instances, creating and managing S3 buckets, configuring IAM users, groups, roles, and policies, monitoring infrastructure using CloudWatch, taking EBS snapshots, performing backup and recovery activities, and troubleshooting infrastructure and networking issues.

I also administer Linux servers by creating users, managing file permissions, monitoring system performance, troubleshooting server issues, and writing Bash scripts to automate routine administrative tasks. I work closely with the development and DevOps teams during application deployments and production releases to ensure high availability and minimal downtime.

One of my strengths is troubleshooting production issues under pressure. I enjoy identifying root causes, implementing permanent fixes, and continuously improving the stability and security of the cloud environment.

I am now looking for a new opportunity where I can enhance my AWS and cloud administration skills, work on larger-scale cloud environments, and contribute to the success of the organization while continuing to grow professionally.

Question 2: What are your current roles and responsibilities as an AWS Administrator?

The interviewer may ask:

- What is your daily routine?
- Which AWS services do you manage?
- How many servers do you handle?
- Do you support production?

Answer:

Yes, I support production, UAT, and development environments. My primary responsibility is to ensure that the AWS infrastructure is secure, available, and performing efficiently.

My daily routine starts with:

- Checking CloudWatch dashboards and alarms for any critical alerts.
- Verifying the health of production EC2 instances.

- Reviewing backup jobs, EBS snapshots, and monitoring storage utilization.
- Checking application availability behind the Application Load Balancer.
- Reviewing support tickets and resolving infrastructure-related issues.

My day-to-day responsibilities include:

- Launching and managing EC2 instances.
- Creating Amazon Machine Images (AMIs) and EBS snapshots for backup and disaster recovery.
- Managing EBS volumes by creating, attaching, resizing, and monitoring them.
- Creating and managing S3 buckets with versioning, lifecycle policies, and bucket policies.
- Managing IAM users, groups, roles, and policies by following the principle of least privilege.
- Configuring VPC components such as public and private subnets, route tables, Internet Gateways, NAT Gateways, Security Groups, and Network ACLs.
- Configuring and monitoring Application Load Balancers and Auto Scaling Groups.
- Managing DNS records using Route 53.
- Monitoring infrastructure with CloudWatch, creating alarms, and responding to notifications.
- Performing Linux server administration, including user management, file permissions, package updates, and service management.
- Troubleshooting production issues related to EC2, networking, storage, and application availability.
- Automating routine administrative tasks using Bash scripts and the AWS CLI.
- Coordinating with development and DevOps teams during application deployments.
- Maintaining documentation for infrastructure changes and standard operating procedures.
- Following security best practices, applying patches, and participating in regular maintenance activities.

Currently, I manage approximately 40–60 EC2 instances across production, UAT, and development environments. The exact number changes based on project requirements and Auto Scaling activities.

Overall, my role is to ensure high availability, security, performance, backup, monitoring, and smooth day-to-day operations of the AWS infrastructure.

Question 3: Describe your AWS environment.

Be ready to answer:

- How many AWS accounts do you manage?
- How many EC2 instances?
- Which regions do you use?
- Which operating systems?
- Production, UAT, and Development environments?

In my current project, we have a multi-environment AWS infrastructure consisting of Production, UAT, and Development environments.

We manage 3 AWS accounts, one each for Development, UAT, and Production, to maintain security and isolate workloads.

Across these environments, we manage approximately 50 EC2 instances:

- Production: Around 30 instances
- UAT: Around 10 instances
- Development: Around 10 instances

Most of our infrastructure is deployed in the Asia Pacific (Mumbai) – ap-south-1 region. We also use another region for backup and disaster recovery, depending on business requirements.

The operating systems we support include:

- Amazon Linux 2
- Red Hat Enterprise Linux (RHEL)
- Ubuntu Server

The main AWS services we manage are:

- EC2 for compute
- EBS for block storage
- S3 for object storage and backups
- IAM for identity and access management
- VPC for networking
- Security Groups and Network ACLs for security
- Application Load Balancer (ALB)
- Auto Scaling Groups
- Route 53 for DNS
- CloudWatch for monitoring and alerts
- AWS CLI for administration and automation

Production Environment:
The production environment hosts live customer applications. We monitor it continuously, respond to incidents, perform regular backups, apply security patches during maintenance windows, and ensure high availability.

UAT Environment:
The UAT environment is used by testers and business users to validate new application releases before they are deployed to production.

Development Environment:
The development environment is used by developers to build, test, and validate new features before they move to UAT.

As an AWS Administrator, my responsibilities include provisioning infrastructure, monitoring system health, troubleshooting issues, managing backups, configuring security, and supporting deployments across all three environments while ensuring minimal downtime and business continuity.

What is an EC2 Instance?

Amazon EC2 (Elastic Compute Cloud) is a web service provided by AWS that allows us to launch and manage virtual servers in the cloud. These virtual servers are called EC2 instances.

EC2 provides scalable and resizable computing capacity, so we can quickly provision servers without purchasing physical hardware.

When creating an EC2 instance, we choose:

- An Amazon Machine Image (AMI), which defines the operating system and software.
- An instance type, such as t3.micro or m5.large, based on CPU and memory requirements.
- A VPC and subnet for networking.
- A Security Group to control inbound and outbound traffic.
- An SSH key pair (for Linux) or an RDP password (for Windows) to access the server.
- An EBS volume for storage.

In my role as an AWS Administrator, I use EC2 instances to host web applications, application servers, databases, and other business services. My responsibilities include launching instances, monitoring their health, resizing them when needed, creating AMIs, taking EBS snapshots, applying patches, and troubleshooting connectivity or performance issues

Interview Answer

Question: How do you launch a new EC2 instance?

To launch a new EC2 instance, I follow these steps:

Step 1: Login to AWS Console

- Log in to the AWS Management Console.
- Navigate to EC2.
- Select the required AWS Region (for example, ap-south-1 (Mumbai)).

Step 2: Launch Instance

- Click Launch Instance.
- Enter a meaningful instance name, such as Web-Server-Prod-01.

Step 3: Choose an AMI (Amazon Machine Image)

Select the operating system based on the application requirement:

- Amazon Linux 2
- Red Hat Enterprise Linux (RHEL)
- Ubuntu Server
- Windows Server

Step 4: Choose an Instance Type

Select the instance type according to CPU and memory requirements.

Examples:

- t3.micro – Small workloads
- t3.medium – Medium applications
- m5.large – Production applications

Step 5: Configure Key Pair

- Select an existing SSH key pair or create a new one.
- This key is used to securely connect to Linux instances using SSH.

Step 6: Configure Network

- Select the appropriate VPC.
- Choose the correct Subnet (public or private).
- Enable or disable Auto-assign Public IP depending on the requirement.

Step 7: Configure Security Group

Configure inbound rules, for example:

Port	Protocol	Purpose
22	SSH	Linux administration
80	HTTP	Web traffic
443	HTTPS	Secure web traffic

Step 8: Configure Storage

- Attach an EBS volume.
- Select the required size (for example, 30 GB or 50 GB).
- Choose the appropriate volume type such as gp3.

Step 9: Review and Launch

- Review all settings.
- Click Launch Instance.

Step 10: Verify the Instance

After launch, I verify:

- Instance state is Running.
- Status checks have passed.
- Security Group is correctly configured.
- Public or private IP address is assigned as expected.
- SSH connectivity works successfully.

Post-Launch Activities

Once the server is running, I typically:

- Connect using SSH.
- Update the operating system.
- Install required packages and application software.
- Attach additional EBS volumes if needed.
- Configure CloudWatch monitoring.
- Assign an IAM role if the instance needs access to AWS services.
- Register the instance with an Application Load Balancer if it's part of a web application.
- Tag the instance with details such as Name, Environment, Project, and Owner.

Interview Tip

The interviewer may ask:

"Why do you attach an IAM Role instead of storing AWS Access Keys on the EC2 instance?"

A good answer is:

"IAM Roles provide temporary credentials that are automatically rotated by AWS, making them more secure than storing long-term access keys on the server. This follows AWS security best practices."

This demonstrates both practical knowledge and awareness of AWS security principles.

An EC2 instance is not reachable via SSH. How do you troubleshoot it?

Expected topics:

- Security Groups
- Network ACLs
- Route Tables
- Internet Gateway
- Public IP/Elastic IP
- SSH key
- SSH service
- Disk space

- Instance status checks

"If an EC2 instance is not reachable via SSH, I first verify that the instance is running and has passed its status checks. Next, I check whether it has a Public IP or Elastic IP, then verify the Security Group allows TCP port 22. After that, I review the Network ACL, Route Table, and Internet Gateway configuration. Finally, I confirm I'm using the correct SSH key and, if I have alternate access, I verify that the SSH service is running on the instance. This step-by-step approach helps me quickly identify and resolve the issue."

Question: What is the difference between EBS and Instance Store?

Amazon EBS (Elastic Block Store) and Instance Store are both storage options for EC2 instances, but they differ in persistence, performance, and use cases.

Feature	Amazon EBS	Instance Store
Storage Type	Network-attached block storage	Storage physically attached to the host server
Data Persistence	Persistent – data remains even if the instance is stopped	Temporary – data is lost if the instance is stopped, terminated, or the host fails
Performance	High performance and consistent	Very high performance with low latency
Backup	Supports EBS Snapshots	Does not support snapshots
Resize	Can increase volume size	Cannot resize independently
Detach/Attach	Can detach and attach to another EC2 instance (within the same Availability Zone)	Cannot be detached or moved
Cost	Charged separately based on volume size and type	Included with supported instance types
Best Use	Operating systems, databases, application data	Temporary data, cache, scratch space, buffers

Amazon EBS (Elastic Block Store)

- Persistent block storage for EC2 instances.
- Data remains even after stopping the instance.
- Supports snapshots for backup.
- Can be encrypted.

- Suitable for production workloads.

Examples:

- Operating system (root volume)
- Database storage
- Business application data
- File systems

Instance Store

- Temporary storage attached to the physical host.
- Offers very fast read/write performance.
- Data is lost if the instance is stopped, terminated, or the underlying host changes.
- Ideal for temporary data only.

Examples:

- Cache files
- Temporary processing data
- Scratch space
- Buffers

Real-Time Example

Suppose I have a production database server. I would use EBS because the data must be persistent and backed up with snapshots.

If I have an application that processes large temporary files or uses a cache, I would use Instance Store because it provides very high performance, and losing the temporary data is acceptable.

Short Interview Answer (30 Seconds)

"EBS is persistent block storage attached over the AWS network. It retains data after an instance is stopped, supports snapshots, encryption, and resizing, making it suitable for operating systems, databases, and production applications. Instance Store is physically attached to the host machine, provides very high-performance temporary storage, but all data is lost if the instance is stopped, terminated, or the host fails. It is mainly used for cache, scratch data, and other temporary workloads."

Interview Answer

Question: How do you increase the root EBS volume from 30 GB to 100 GB without downtime?

Amazon EBS supports Elastic Volumes, which allows us to increase the size of an EBS volume while the EC2 instance is running. There is no need to stop or reboot the instance in most cases.

Step 1: Identify the EC2 Instance

- Go to the EC2 Console.
- Select the EC2 instance.
- Under the Storage tab, identify the root EBS volume (for example, `/dev/xvda`).

Step 2: Modify the EBS Volume

- Click the Volume ID.
- Select Actions → Modify Volume.
- Change the size from 30 GB to 100 GB.
- Click Modify and confirm the change.

AWS increases the EBS volume in the background while the instance continues running.

Step 3: Verify the Volume Modification

- Wait until the volume status changes to Optimizing or Completed.
- The EC2 instance remains available during this process.

Step 4: Extend the File System

Increasing the EBS volume only increases the disk size. The operating system still sees the old partition size, so the file system must be extended.

For Amazon Linux/RHEL (XFS file system)

Check the disk:

Bash

```
lsblk
```

Grow the partition:

Bash

```
sudo growpart /dev/xvda 1
```

Extend the XFS file system:

Bash

```
sudo xfs_growfs -d /
```

For Ubuntu (EXT4 file system)

Grow the partition:

Bash

```
sudo growpart /dev/xvda 1
```

Resize the EXT4 file system:

Bash

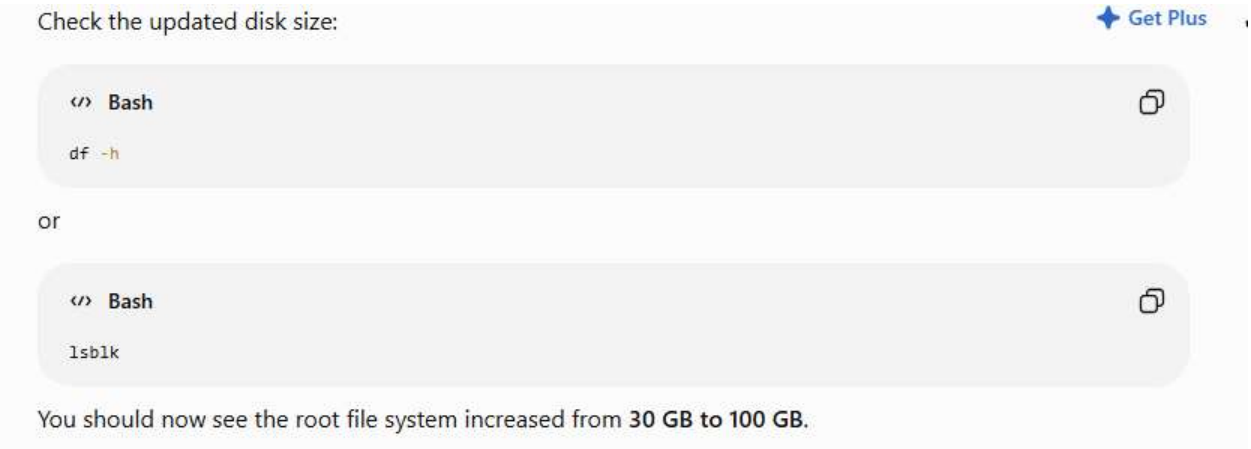
```
sudo resize2fs /dev/xvda1
```

Step 5: Verify the New Size

Check the updated disk size:

Bash

```
df -h
```



Short Interview Answer (1 Minute)

"To increase the root EBS volume from 30 GB to 100 GB without downtime, I go to the EC2 console, select the attached EBS volume, choose Modify Volume, and increase its size to 100 GB. After AWS completes the volume expansion, I log in to the EC2 instance and extend the partition and file system using commands such as growpart and xfs_growfs for XFS, or resize2fs for EXT4. Finally, I verify the new size using df -h or lsblk. The instance remains running throughout the process."

Common Follow-up Questions

Q1. Can you decrease an EBS volume from 100 GB back to 30 GB?

Answer:

No. AWS allows you to increase the size of an EBS volume, but it does not allow decreasing it. To reduce the size, you typically create a new smaller volume, copy the data, and replace the original volume.

Q2. Does the EC2 instance need to be restarted?

Answer:

No. In most cases, the volume can be expanded while the EC2 instance is running. After the volume modification, only the partition and file system need to be extended inside the operating system.

Interview Tip

A strong interview response always mentions both parts of the process:

1. Increase the EBS volume in AWS.
2. Extend the partition and file system inside the operating system.

Many candidates only describe the AWS console steps and forget the second part, which is essential for a complete answer.

Amazon S3 (Simple Storage Service) is a cloud-based object storage service offered by Amazon Web Services (AWS). It lets you store and retrieve virtually any amount of data over the internet.

What can you use Amazon S3 for?

- Storing files such as photos, videos, documents, and backups.
- Hosting static websites (HTML, CSS, JavaScript).
- Data lakes and analytics for processing large datasets.
- Application storage for web and mobile apps.
- Disaster recovery by keeping backups in the cloud.

How it works

S3 stores data as objects inside buckets:

- Bucket: A container for your data (similar to a top-level folder).
- Object: A file plus its metadata (such as an image, PDF, or video).
- Each object has a unique key (its name/path within the bucket).

For example:

```
Bucket: my-company-backups
 ├── database-backup.sql
 ├── images/
 │    ├── logo.png
 │    └── banner.jpg
 └── reports/
      └── sales-2026.pdf
```

Key features

- Highly durable: Designed for 99.999999999% (11 nines) data durability.
- Scalable: Store from a few files to billions of objects.
- Secure: Supports encryption, access control, and fine-grained permissions.
- Cost-effective: Pay only for the storage and requests you use.

- Versioning: Keep multiple versions of the same file to recover from accidental deletions or overwrites.

Simple example

Imagine you're building a photo-sharing app:

1. A user uploads a photo.
2. The app stores the photo in an S3 bucket.
3. When someone views the photo, the app retrieves it from S3.

In this setup, your application doesn't need to store large image files on its own server.

You can learn more in the official documentation: Amazon S3 Documentation.

If you're new to cloud computing, think of Amazon S3 as a highly reliable online hard drive that applications can access programmatically from anywhere.

What is Amazon S3?

Amazon Simple Storage Service (Amazon S3) is an object storage service offering industry-leading scalability, data availability, security, and performance. Millions of customers of all sizes and industries store, manage, analyze, and protect any amount of data for virtually any use case, such as data lakes, cloud-native applications, and mobile apps. With cost-effective storage classes and easy-to-use management features, you can optimize costs, organize and analyze data, and configure fine-tuned access controls to meet specific business and compliance requirements.

Scalability

> You can store virtually any amount of data with S3 all the way to exabytes with unmatched performance. S3 is fully elastic, automatically growing and shrinking as you add and remove data. There’s no need to provision storage, and you pay only for what you use.

Durability and availability

> Amazon S3 provides the most durable storage in the cloud and industry leading availability. Based on its unique architecture, S3 is designed to provide 99.999999999% (11 nines) data durability and 99.99% availability by default, backed by the strongest SLAs in the cloud.

Security and data protection

Protect your data with unmatched security, data protection, compliance, and access control capabilities. S3 is secure, private, and encrypted by default, and also supports numerous auditing capabilities to monitor access requests to your S3 resources.

Lowest price and highest performance

S3 delivers multiple storage classes with the best price performance for any workload and automated data lifecycle management, so you can store massive amounts of frequently, infrequently, or rarely accessed data in a cost-efficient way. S3 delivers the resiliency, flexibility, latency, and throughput, to ensure storage never limits performance.

What are Amazon S3 Storage Classes?

Amazon S3 Storage Classes are different storage options in Amazon S3 designed for different access patterns, performance needs, and costs. Choosing the right storage class helps reduce storage costs while maintaining the required availability and retrieval speed.

Interview Definition (1 Minute)

"Amazon S3 Storage Classes are different types of storage provided by AWS to optimize cost based on how frequently data is accessed. Frequently accessed data can be stored in S3 Standard, while infrequently accessed or archived data can be stored in lower-cost storage classes such as Standard-IA, One Zone-IA, Glacier Instant Retrieval, Glacier Flexible Retrieval, Glacier Deep Archive, or Intelligent-Tiering."

Amazon S3 Storage Classes

Storage Class	Best For	Retrieval Time	Cost
S3 Standard	Frequently accessed data	Immediate	Highest
S3 Intelligent-Tiering	Unknown or changing access patterns	Immediate	Automatically optimizes cost
S3 Standard-IA	Infrequently accessed data	Immediate	Lower than Standard
S3 One Zone-IA	Infrequently accessed, non-critical data	Immediate	Cheaper than Standard-IA
S3 Glacier Instant Retrieval	Rarely accessed data needing instant access	Immediate	Very Low

Storage Class	Best For	Retrieval Time	Cost
S3 Glacier Flexible Retrieval	Long-term backups	Minutes to Hours	Very Low
S3 Glacier Deep Archive	Long-term archival	12–48 Hours	Lowest

---

1. S3 Standard

- Used for frequently accessed data.
- High availability and durability.
- Best for websites, mobile apps, and content distribution.
- Data is available immediately.

Example: Website images, videos, application files.

---

2. S3 Intelligent-Tiering

- Automatically moves data between storage tiers based on access patterns.
- No performance impact.
- Saves costs when you don't know how often data will be accessed.

Example: User-uploaded documents with unpredictable access.

---

3. S3 Standard-IA (Infrequent Access)

- For data accessed occasionally.
- Lower storage cost than Standard.
- Retrieval is immediate, but retrieval charges apply.

Example: Monthly backups, disaster recovery files.

---

4. S3 One Zone-IA

- Stores data in only one Availability Zone.
- Lower cost than Standard-IA.
- Suitable for data that can be recreated if lost.

Example: Secondary backups, development environment data.

---

5. S3 Glacier Instant Retrieval

- Archive storage with instant access.
- Very low storage cost.
- Good for data that is rarely accessed but must be retrieved immediately.

Example: Medical records, media archives.

---

6. S3 Glacier Flexible Retrieval

- Designed for long-term backups.
- Retrieval takes from minutes to several hours.
- Lower storage cost than Glacier Instant Retrieval.

Example: Yearly database backups.

---

7. S3 Glacier Deep Archive

- Lowest-cost S3 storage class.
- Retrieval takes 12 to 48 hours.
- Best for compliance and long-term data retention.

Example: Financial records, legal documents retained for many years.

Easy Memory Trick for Interviews

Frequency	Storage Class
Daily access	S3 Standard
Access pattern unknown	Intelligent-Tiering
Monthly access	Standard-IA
Non-critical monthly access	One Zone-IA
Rare but instant access	Glacier Instant Retrieval
Rare backups	Glacier Flexible Retrieval
Never accessed unless needed	Glacier Deep Archive

Interview Question

Q: Which S3 storage class would you choose for storing 10-year-old company records that are rarely accessed?

Answer:
S3 Glacier Deep Archive, because it offers the lowest storage cost and is ideal for long-term archival data that is rarely retrieved.

Interview Summary (2-Minute Answer)

"Amazon S3 provides multiple storage classes to optimize storage cost based on access frequency. S3 Standard is used for frequently accessed data. Intelligent-Tiering automatically moves data between tiers based on usage. Standard-IA and One Zone-IA are suitable for infrequently accessed data. Glacier Instant Retrieval provides instant access to archived data, while Glacier Flexible Retrieval is designed for long-term backups with retrieval in minutes or hours. Glacier Deep Archive is the most cost-effective option for data that must be retained for years and is rarely accessed. All storage classes provide high durability, but they differ in availability, retrieval time, and cost."

1. S3 Versioning

Definition

S3 Versioning is a feature that keeps multiple versions of an object in the same S3 bucket. If a file is modified or deleted accidentally, you can restore a previous version.

Interview Answer (1 Minute)

"Amazon S3 Versioning is used to maintain multiple versions of an object in an S3 bucket. It protects data from accidental deletion or overwrites. When versioning is enabled, every upload of the same object creates a new version with a unique Version ID. We can restore older versions whenever required."

Example

Suppose you upload a file:

```
report.pdf  (Version 1)
```

Later, you upload an updated file with the same name:

```
report.pdf  (Version 2)
```

Both versions are stored.

```
report.pdf
├── Version 1
└── Version 2 (Latest)
```

If Version 2 is deleted accidentally, Version 1 can still be restored.

Benefits

- Protects against accidental deletion.
- Protects against accidental overwriting.
- Helps recover previous versions.
- Supports backup and disaster recovery.
- Useful for auditing and compliance.

Important Points

- Versioning is enabled **at the bucket level**.
- Once enabled, it **cannot be completely disabled** (it can only be suspended).
- Every object version has a unique **Version ID**.
- Keeping multiple versions increases storage usage and cost.

2. S3 Lifecycle Policies

Definition

Lifecycle Policies automatically move objects between storage classes or delete them after a specified period, helping reduce storage costs.

Interview Answer (1 Minute)

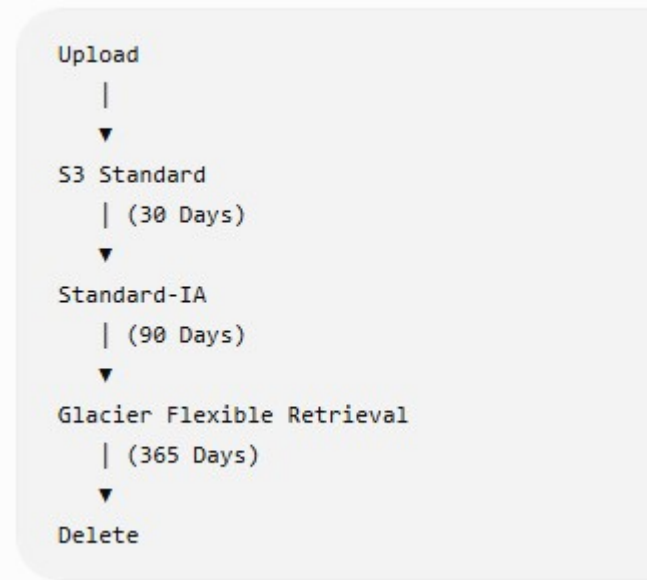
"Amazon S3 Lifecycle Policies automate storage management by moving objects to cheaper storage classes after a defined number of days or deleting them when they are no longer needed. This helps optimize storage costs without manual intervention."

Example

Suppose your application stores log files.

Lifecycle Rule:

- Day 0 → Store in **S3 Standard**
- After 30 days → Move to **S3 Standard-IA**
- After 90 days → Move to **S3 Glacier Flexible Retrieval**
- After 365 days → Delete the object



Benefits

- Reduces storage costs.
- Automatically archives old data.
- Automatically deletes expired files.
- No manual storage management.
- Supports long-term retention policies.

Common Lifecycle Actions

- Transition from Standard to Standard-IA.
- Transition to Glacier storage classes.
- Permanently delete expired objects.

- Delete incomplete multipart uploads after a specified number of days.

Versioning vs Lifecycle Policies

Feature	Versioning	Lifecycle Policies
Purpose	Protect data	Reduce storage cost
Function	Keeps multiple versions	Moves or deletes objects automatically
Recovery	Yes, previous versions can be restored	No, once an object is permanently deleted, it cannot be recovered
Configuration	Enabled at the bucket level	Configured through lifecycle rules
Benefit	Prevents data loss	Optimizes storage costs

Real-Time Example

A company stores employee salary reports.

- **Versioning:** If someone accidentally replaces `salary.xlsx`, the previous version can be restored.
- **Lifecycle Policy:** After one year, move the file to Glacier for cheaper storage. After seven years (depending on company policy), delete it automatically.

Interview Question

Q: Why do we use Versioning and Lifecycle Policies together?

Answer:

"Versioning protects data by maintaining multiple versions of objects, while Lifecycle Policies automatically transition older object versions to lower-cost storage classes or delete them after a retention period. Together, they improve data protection and reduce storage costs."

2-Minute Interview Summary

"Amazon S3 Versioning protects data by keeping multiple versions of the same object, allowing recovery from accidental deletion or overwrites. Each version has a unique Version ID, and versioning is enabled at the bucket level. S3 Lifecycle Policies automate storage management by moving objects to more cost-effective storage classes, such as Standard-IA or Glacier, and

deleting objects when they are no longer needed. Versioning improves data protection, while Lifecycle Policies help optimize storage costs."

How do you secure an Amazon S3 bucket? (Interview Answer)

Interview Answer (2 Minutes)

"To secure an Amazon S3 bucket, I follow the principle of least privilege by granting only the permissions that users or applications require. I keep the bucket private by default, use IAM policies and bucket policies to control access, enable Block Public Access to prevent accidental exposure, encrypt data both at rest and in transit, enable Versioning to protect against accidental deletion, and enable logging and monitoring with CloudTrail and S3 server access logs. I also use lifecycle policies to manage data efficiently and regularly review permissions."

Best Practices to Secure an S3 Bucket

1. Block Public Access (Most Important)

- Enable **Block Public Access** for the bucket.
- Prevents accidental public exposure of sensitive data.
- Keep buckets private unless public access is explicitly required.

Interview Tip: Most security incidents occur because a bucket was unintentionally made public.

---

2. Use IAM Policies

- Grant permissions only to authorized users, groups, or roles.
- Follow the **Principle of Least Privilege**.
- Avoid giving unnecessary permissions like `s3:*`.

Example:

- Application → Read and Write
- Developers → Read Only
- Administrators → Full Access

---

3. Use Bucket Policies

- Control access at the bucket level.
- Allow or deny access to specific AWS accounts, IAM roles, or IP addresses.
- Restrict access to trusted users or networks.

---

4. Enable Encryption

Encrypt data:

At Rest

- **SSE-S3** (Amazon-managed keys)
- **SSE-KMS** (AWS KMS keys with additional access control)
- **SSE-C** (Customer-provided encryption keys)

In Transit

- Require **HTTPS (SSL/TLS)** for all requests.

---

5. Enable Versioning

- Protects against accidental deletion or overwriting.
- Allows recovery of previous versions of objects.

---

6. Enable Logging and Monitoring

Use:

- **AWS CloudTrail** → Records API calls.
- **S3 Server Access Logs** → Records bucket access.
- **Amazon CloudWatch** → Monitors metrics and creates alarms.

This helps with auditing and troubleshooting.

---

7. Enable MFA Delete (for highly sensitive data)

- Requires **Multi-Factor Authentication (MFA)** before permanently deleting object versions or changing versioning settings.
- Adds an extra layer of protection.

---

8. Use Lifecycle Policies

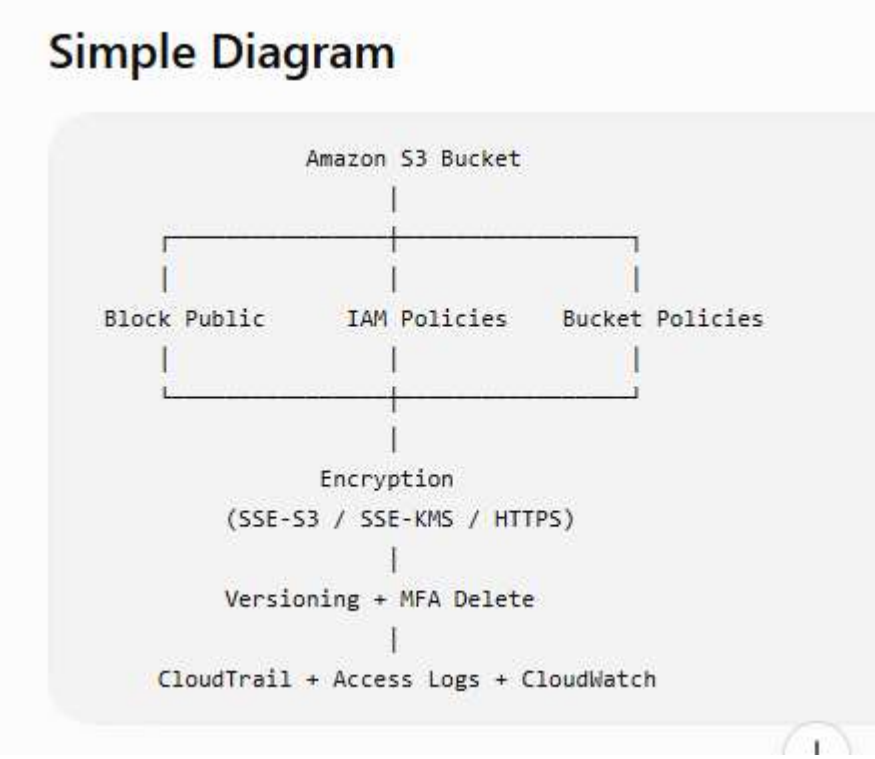
- Automatically move old data to cheaper storage classes.
- Delete unnecessary data after the required retention period.

- Helps reduce storage costs while maintaining compliance.

---

9. Regularly Review Permissions

- Remove unused users and roles.
- Audit bucket policies and IAM policies.
- Avoid overly permissive permissions.



nterview Question

Q: What are the most important ways to secure an S3 bucket?

Answer:

1. Enable **Block Public Access**.
2. Use **IAM policies** with least privilege.
3. Apply **bucket policies** where needed.
4. Enable **encryption** (SSE-S3 or SSE-KMS and HTTPS).
5. Enable **Versioning**.
6. Monitor access with **CloudTrail**, **S3 Access Logs**, and **CloudWatch**.
7. Enable **MFA Delete** for critical buckets.

Interview Summary (2 Minutes)

"To secure an S3 bucket, I first keep it private by enabling Block Public Access. I use IAM and bucket policies to provide only the required permissions following the principle of least privilege. I encrypt data at rest using SSE-S3 or SSE-KMS and enforce HTTPS for data in transit. I enable Versioning and, for highly sensitive data, MFA Delete to protect against accidental or malicious deletion. Finally, I enable CloudTrail, S3 access logs, and CloudWatch for auditing and monitoring, and I regularly review permissions to maintain security."

☆ Interview Tip

If you're asked **"What are the top three security features for an S3 bucket?"**, a strong answer is:

1. **Block Public Access**
2. **IAM Policies and Bucket Policies (Least Privilege)**
3. **Encryption (SSE-KMS) and HTTPS**

These are the features interviewers most often expect candidates to mention first.

What is IAM? (AWS Interview Answer)

Definition

IAM (Identity and Access Management) is an AWS service that helps you **securely manage access** to AWS resources. It allows you to create users, groups, and roles, and control what actions they can perform using policies.

Interview Answer (1 Minute)

"AWS IAM (Identity and Access Management) is a global AWS service used to securely control access to AWS resources. It enables us to create users, groups, and roles, and assign permissions through IAM policies. IAM follows the Principle of Least Privilege, meaning users receive only the permissions they need to perform their tasks. It also supports Multi-Factor Authentication (MFA) and temporary credentials for enhanced security."

IAM Components

1. IAM Users

- Represents an individual person or application.
- Each user has unique login credentials.
- Permissions are assigned through policies.

Example:

- Azeedh (Linux Admin)
- Ravi (Developer)
- Priya (Tester)

---

2. IAM Groups

- A collection of IAM users.
- Permissions assigned to the group are inherited by all members.

Example:

```
Developers Group
  ├── User1
  ├── User2
  └── User3

Policy: EC2 Full Access
```

3. IAM Roles

- Used to grant **temporary permissions**.
- No long-term username or password.
- Commonly used by AWS services, applications, or users from another AWS account.

Example:

- EC2 instance accessing an S3 bucket.
- Lambda function accessing DynamoDB.

---

4. IAM Policies

- JSON documents that define permissions.
- Specify:
    - **Effect** (Allow or Deny)
    - **Action** (What operations are allowed)
    - **Resource** (Which AWS resource)

Example:

JSON

```
{
  "Effect": "Allow",
  "Action": "s3:GetObject",
  "Resource": "arn:aws:s3:::my-bucket/*"
}
```

This policy allows reading objects from the specified S3 bucket.

IAM Best Practices

- ✅ Follow the **Principle of Least Privilege**.
- ✅ Enable **Multi-Factor Authentication (MFA)**.
- ✅ Avoid using the **root account** for daily tasks.
- ✅ Use **IAM Roles** instead of storing access keys on EC2 instances.
- ✅ Rotate access keys regularly.
- ✅ Review and remove unused users and permissions.

Real-Time Example

Suppose a company has three teams:

Team	Permission
Developers	EC2 and S3 access
Testers	Read-only access
Administrators	Full AWS access

Instead of assigning permissions to each user individually:

- Create **Developers**, **Testers**, and **Administrators** groups.
- Attach the required policies to each group.
- Add users to the appropriate group.

This simplifies permission management.

Interview Question

Q: What is the difference between an IAM User and an IAM Role?

IAM User	IAM Role
Permanent identity	Temporary identity
Has username and credentials	No permanent credentials
Used by people or applications	Used by AWS services, applications, or cross-account access
Long-term access	Temporary access

Interview Summary (2 Minutes)

"AWS IAM is a global service used to securely manage access to AWS resources. It allows us to create users, groups, and roles, and assign permissions through IAM policies. Users represent individuals, groups simplify permission management for multiple users, roles provide temporary credentials for AWS services and applications, and policies define what actions are allowed or denied on specific resources. IAM also supports MFA and follows the Principle of Least Privilege, making it one of the core security services in AWS."

☆ Interview Tip

A common follow-up question is: **"Is IAM a regional or global service?"**

Answer:
IAM is a Global Service. You create IAM users, groups, roles, and policies once, and they can be used to manage access across AWS services in all regions.

Interview Question: What is the difference between an IAM User?

Get Plus Share

Usually, interviewers ask:

> **"What is the difference between an IAM User, IAM Group, IAM Role, and IAM Policy?"**

Here's a clear interview-ready answer.

Feature	IAM User	IAM Group	IAM Role	IAM Policy
Purpose	Represents an individual person or application	Collection of IAM users	Temporary identity with permissions	JSON document that defines permissions
Credentials	Has username, password, and/or access keys	No credentials	Temporary security credentials	No credentials
Used By	Employees, developers, administrators	Teams (Developers, Testers, Admins)	AWS services, applications, cross-account users	Attached to users, groups, or roles
Permissions	Directly attached or inherited from a group	Permissions shared by all users in the group	Permissions assumed temporarily	Defines Allow or Deny actions
Duration	Long-term	Long-term	Temporary	Not applicable

1. IAM User

- Represents a single person or application.
- Has permanent credentials.
- Can log in to the AWS Management Console or use the AWS CLI/API.

Example:

- Azeedh (Linux Admin)
- Ravi (Developer)

---

2. IAM Group

- A collection of IAM users.
- Makes permission management easier.
- Users inherit the group's permissions.

Example:

```
Developers Group
 ├── Ravi
 ├── Azeedh
 └── Kumar

Policy: EC2 Full Access
```

3. IAM Role

- Provides temporary permissions.
- Does not have a username or password.
- Assumed by AWS services or trusted users.

Example:

- An EC2 instance assumes a role to access an S3 bucket.
- A Lambda function assumes a role to write logs to CloudWatch.

---

4. IAM Policy

- A JSON document that defines permissions.
- Specifies:
    - **Effect** (Allow or Deny)
    - **Action** (What operations are allowed)
    - **Resource** (Which AWS resource)

Example:

JSON

```
{
  "Effect": "Allow",
  "Action": "s3:GetObject",
  "Resource": "arn:aws:s3:::my-bucket/*"
}
```

Easy Memory Trick

- **User** → One Person
- **Group** → Many Users
- **Role** → Temporary Access
- **Policy** → Permission Rules

Real-Time Example

A company has:

- **Azeedh** → IAM User
- **Developers** → IAM Group
- **EC2 Server** → IAM Role (to access S3)
- **AmazonS3ReadOnlyAccess** → IAM Policy

The EC2 server uses the IAM Role to access S3 without storing access keys.

2-Minute Interview Answer

"An IAM User is a permanent identity for an individual with login credentials. An IAM Group is a collection of users that share the same permissions. An IAM Role provides temporary permissions and is commonly used by AWS services such as EC2 or Lambda, or for cross-account access. An IAM Policy is a JSON document that defines what actions are allowed or denied on AWS resources. Users, groups, and roles receive their permissions through IAM policies."

Interview Tip

If the interviewer asks **"When would you use an IAM Role instead of an IAM User?"**, answer:

"I use an IAM Role when an AWS service, such as an EC2 instance or Lambda function, needs to access other AWS resources securely. Roles provide temporary credentials and eliminate the need to store long-term access keys."

Interview Question: How do you provide temporary access to an EC2 instance?

Interview Answer (2 Minutes)

"The recommended way to provide temporary access to an EC2 instance is by using an IAM Role. I create an IAM Role with only the required permissions, attach the role to the EC2 instance, and AWS automatically provides temporary security credentials through the Instance Metadata Service (IMDS). The application running on the EC2 instance uses these temporary credentials, which are automatically rotated by AWS. This is more secure than storing long-term access keys on the instance."

Steps

Step 1: Create an IAM Role

- Open the IAM console.
- Create a role.
- Choose **AWS Service** as the trusted entity.
- Select **EC2**.

Step 2: Attach a Policy

Attach only the permissions the EC2 instance needs.

Examples:

- AmazonS3ReadOnlyAccess
- AmazonS3FullAccess
- CloudWatchAgentServerPolicy

Step 3: Attach the Role to the EC2 Instance

- Go to **EC2 Console**
- Select the EC2 instance
- **Actions → Security → Modify IAM Role**
- Choose the IAM role
- Save the changes

Step 4: AWS Provides Temporary Credentials

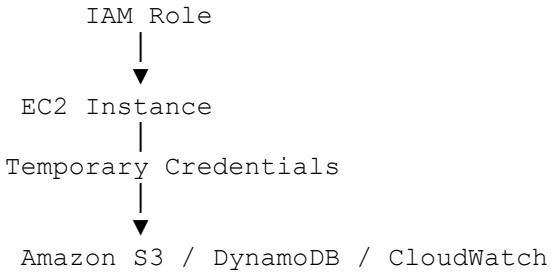
AWS automatically generates:

- Temporary Access Key
- Secret Access Key
- Session Token

These credentials:

- Are short-lived
- Rotate automatically
- Do not need to be stored in your application

Architecture Diagram



Why Use IAM Roles?

✅ No hardcoded AWS access keys

✅ Temporary credentials

✅ Credentials rotate automatically

✅ More secure

✅ Follows AWS security best practices

Wrong Approach (Avoid)

❌ Creating an IAM user

❌ Storing Access Key and Secret Key on the EC2 instance

Problems:

- Keys can be stolen.
- Manual key rotation is required.
- Higher security risk.

Real-Time Example

Suppose your application on an EC2 instance needs to upload log files to an S3 bucket.

Instead of storing AWS access keys:

1. Create an IAM role with S3 write permissions.
2. Attach the role to the EC2 instance.
3. The application automatically uses temporary credentials to upload files securely.

Interview Follow-Up Questions

Q1: Why use an IAM Role instead of an IAM User?

Answer:

- IAM User uses long-term credentials.
- IAM Role provides temporary credentials that AWS rotates automatically.
- IAM Roles are more secure and are the AWS-recommended approach for EC2 instances.

Q2: Where does the EC2 instance get the temporary credentials?

Answer:

- From the **Instance Metadata Service (IMDS)**.
- Applications and AWS SDKs retrieve the temporary credentials automatically from IMDS.

2-Minute Interview Summary

"To provide temporary access to an EC2 instance, I create an IAM Role with the required permissions and attach it to the instance. AWS automatically generates temporary security credentials and makes them available through the Instance Metadata Service. These credentials are automatically rotated and used by AWS SDKs without storing long-term access keys on the server. This follows the Principle of Least Privilege and is the recommended AWS security best practice."

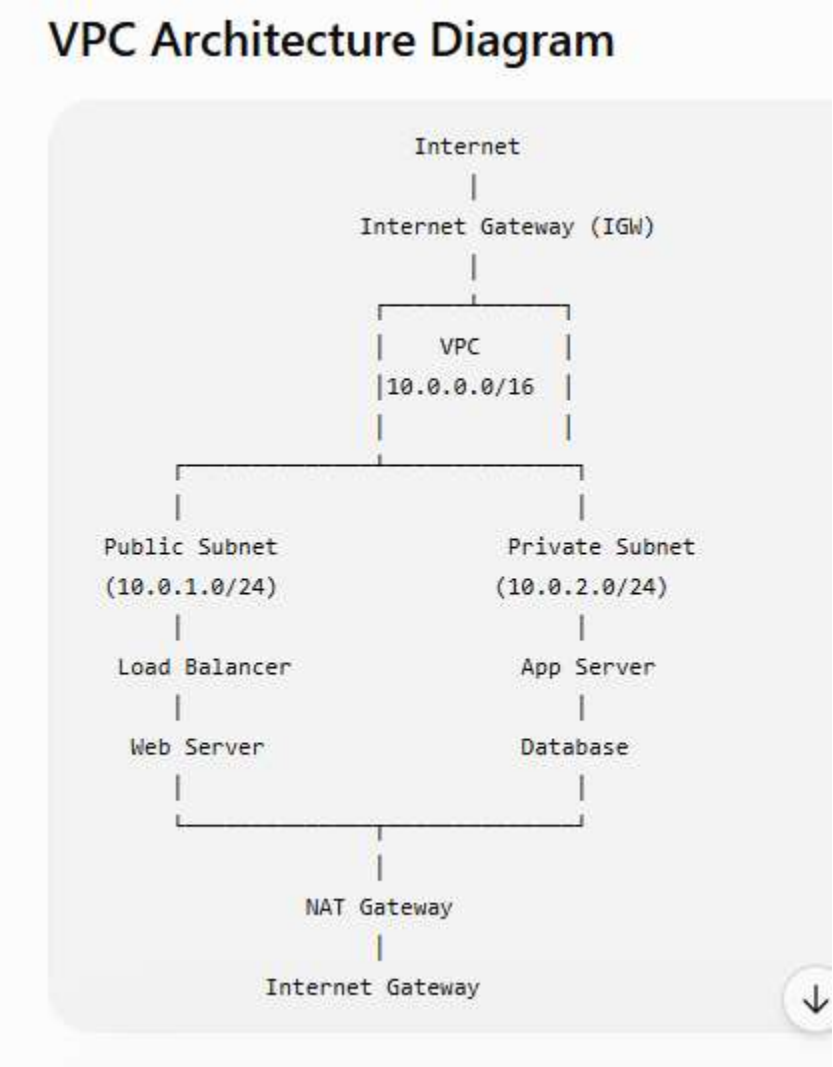
Explain the Architecture of an Amazon VPC (Interview Answer)

What is a VPC?

Amazon VPC (Virtual Private Cloud) is a logically isolated virtual network in AWS where you can launch and manage AWS resources such as EC2 instances, databases, and load balancers. It gives you complete control over networking, including IP addressing, subnets, routing, and security.

Interview Answer (1 Minute)

"An Amazon VPC is a private virtual network in AWS that allows us to securely deploy AWS resources. A VPC contains subnets, route tables, an Internet Gateway, NAT Gateway, security groups, network ACLs, and optionally a Virtual Private Gateway for VPN connectivity. Public subnets host internet-facing resources like web servers, while private subnets host backend applications and databases. Route tables control traffic flow, Security Groups act as instance-level firewalls, and Network ACLs provide subnet-level security."



Components of a VPC

1. VPC

- The virtual network that contains all AWS resources.
- Example CIDR block: **10.0.0.0/16**

---

2. Subnets

A subnet is a smaller network inside a VPC.

Public Subnet

- Has a route to the Internet Gateway.
- Used for:

- Web Servers
- Load Balancers
- Bastion Hosts

Private Subnet

- No direct internet access.
- Used for:
  - Application Servers
  - Databases
  - Internal services

---

3. Internet Gateway (IGW)

- Enables communication between the VPC and the internet.
- Attached to only one VPC.
- Required for public internet access.

---

4. NAT Gateway

- Allows instances in private subnets to access the internet for outbound traffic.
- Prevents inbound internet connections.

Example:

- Download software updates.
- Install application packages.

---

5. Route Table

- Determines where network traffic is directed.

Example:

Destination	Target
10.0.0.0/16	Local
0.0.0.0/0	Internet Gateway (Public Subnet)
0.0.0.0/0	NAT Gateway (Private Subnet)

---

6. Security Groups

- Instance-level virtual firewall.
- Controls inbound and outbound traffic.
- **Stateful** (return traffic is automatically allowed).

Example:

- Allow SSH (22) from your office IP.
- Allow HTTP (80).
- Allow HTTPS (443).

---

7. Network ACL (NACL)

- Subnet-level firewall.
- Controls inbound and outbound traffic.
- **Stateless** (return traffic must be explicitly allowed).

---

8. Elastic IP

- Static public IPv4 address.
- Commonly used with:
    - NAT Gateway
    - Bastion Host

---

9. VPN Gateway (Optional)

- Connects an on-premises data center to AWS over a secure VPN.
- Used in hybrid cloud environments.

Real-Time Example

Imagine an e-commerce application:

- **Public Subnet**
    - Application Load Balancer
    - Web Server
- **Private Subnet**
    - Application Server
    - Database (RDS)

Flow:

```
User
  |
Internet
  |
Internet Gateway
  |
Load Balancer
  |
Web Server
  |
Application Server
  |
Database
```

The database remains in a private subnet and is not directly accessible from the internet.

Common Interview Questions

Q1: What is the difference between a Public and Private Subnet?

Public Subnet	Private Subnet
Has a route to the Internet Gateway	No direct route to the Internet Gateway
Internet accessible	Not internet accessible directly
Hosts web servers, load balancers	Hosts application servers and databases

---

Q2: Why do we use a NAT Gateway?

Answer:

A NAT Gateway allows instances in private subnets to access the internet for outbound traffic, such as downloading updates, without exposing them to inbound internet traffic.

Q3: What is the difference between a Security Group and a Network ACL?

Security Group	Network ACL
Instance-level firewall	Subnet-level firewall

Security Group	Network ACL
Stateful	Stateless
Supports Allow rules only	Supports Allow and Deny rules

2-Minute Interview Summary

"An Amazon VPC is a logically isolated virtual network in AWS. It contains public and private subnets, route tables, an Internet Gateway for public access, and a NAT Gateway for outbound internet access from private subnets. Security Groups provide stateful instance-level security, while Network ACLs provide stateless subnet-level security. Route tables determine where traffic is directed. This architecture enables secure, scalable, and highly available cloud applications by exposing only the necessary components to the internet while keeping backend servers and databases protected."

Difference Between Security Group and Network ACL (NACL)

This is one of the **most frequently asked AWS interview questions**.

Interview Answer (2 Minutes)

"Both Security Groups and Network ACLs are used to control network traffic in a VPC. A Security Group acts as a stateful firewall at the EC2 instance level, while a Network ACL acts as a stateless firewall at the subnet level. Security Groups support only Allow rules, whereas Network ACLs support both Allow and Deny rules. Security Groups automatically allow return traffic, but with Network ACLs, return traffic must be explicitly allowed."

Security Group vs Network ACL

Feature	Security Group	Network ACL (NACL)
Level	Instance-level firewall	Subnet-level firewall
Works On	EC2 instances	Entire subnet

Feature	Security Group	Network ACL (NACL)
Type	**Stateful**	**Stateless**
Rules	Allow rules only	Allow and Deny rules
Return Traffic	Automatically allowed	Must be explicitly allowed
Default Behavior	Denies all inbound, allows all outbound (default SG)	Default NACL allows all inbound and outbound
Evaluation	All rules are evaluated	Rules are processed in order (lowest rule number first)
Multiple Attachments	One Security Group can be attached to multiple instances	One NACL can be associated with multiple subnets

---

1. Security Group

Features

- Protects individual EC2 instances.
- Acts as a **stateful firewall**.
- Supports **Allow** rules only.
- Return traffic is automatically permitted.

Example

Allow:

- SSH (22)
- HTTP (80)
- HTTPS (443)

```
Internet
    |
Security Group
    |
EC2 Instance
```

If you allow SSH from your IP address, the response traffic is automatically allowed.

---

2. Network ACL (NACL)

Features

- Protects an entire subnet.
- Acts as a **stateless firewall**.
- Supports both **Allow** and **Deny** rules.
- Return traffic requires explicit rules.

Example

Inbound:

- Allow HTTP (80)
- Deny Telnet (23)

Outbound:

- Allow ephemeral ports for return traffic

```
Internet
    |
Network ACL
    |
Subnet
    |
EC2 Instances
```

Stateful vs Stateless

Security Group (Stateful)

```
Client
   |
SSH (22)
   ▼
EC2 Instance

Response ✔ Automatically Allowed
```

You only need to allow inbound SSH. The response is automatically permitted.

Network ACL (Stateless)

```
Client
   |
SSH (22)
   ▼
```

```
Subnet

Response ✕ Blocked unless outbound rule allows it
```

You must configure both inbound and outbound rules.

Real-Time Example

Imagine an e-commerce application.

Security Group

- Web Server
  - Allow HTTP (80)
  - Allow HTTPS (443)
  - Allow SSH (22) only from the administrator's IP

Network ACL

- Allow all web traffic to the public subnet
- Explicitly deny traffic from a known malicious IP range (if required)

Easy Memory Trick

Security Group	Network ACL
S = Server (Instance)	**N = Network (Subnet)**
S = Stateful	**N = Not Stateful (Stateless)**

Interview Questions

Q1: Which is more secure?

Answer:

Both are important and work together. Security Groups provide instance-level protection, while Network ACLs provide subnet-level protection. Using both gives a stronger security posture.

Q2: Can a Security Group deny traffic?

Answer:

No. Security Groups support only **Allow** rules. Any traffic not explicitly allowed is denied by default.

Q3: Can a Network ACL deny traffic?

Answer:

Yes. Network ACLs support both **Allow** and **Deny** rules.

Interview Summary (2 Minutes)

"A Security Group is a stateful firewall that protects EC2 instances and supports only Allow rules. Return traffic is automatically allowed. A Network ACL is a stateless firewall that protects an entire subnet and supports both Allow and Deny rules. With a Network ACL, inbound and outbound rules must both be configured because return traffic is not automatically allowed. In AWS, Security Groups provide instance-level security, while Network ACLs provide subnet-level security, and both are commonly used together to secure applications."

☆ Interview Tip

If an interviewer asks:

"Which one should I check first if I can't SSH to an EC2 instance?"

A good troubleshooting order is:

1. **Security Group** – Is inbound **TCP port 22** allowed from your IP?
2. **Network ACL** – Are inbound and outbound rules allowing SSH and return traffic?
3. **Route Table** – Does the subnet have the correct route?
4. **Internet Gateway** – Is an IGW attached (for a public instance)?
5. **EC2 Instance** – Is it running, does it have a public IP/Elastic IP, and is the SSH service running?

Public Subnet vs Private Subnet (AWS Interview Answer)

This is one of the **most common AWS interview questions**.

Interview Answer (2 Minutes)

"A subnet is a range of IP addresses within a VPC. The main difference between a Public Subnet and a Private Subnet is internet accessibility. A Public Subnet has a route to an Internet Gateway, allowing resources such as web servers and load balancers to communicate directly with the internet. A Private Subnet does not have a direct route to the Internet Gateway, making it suitable for application servers and databases. If resources in a Private Subnet need outbound internet access, they use a NAT Gateway."

Public Subnet vs Private Subnet

Feature	Public Subnet	Private Subnet
Internet Access	Yes	No (direct access)
Route Table	Route to Internet Gateway (IGW)	No route to IGW
Internet Gateway	Required	Not used directly
NAT Gateway	Not required	Used for outbound internet access
Public IP	Usually assigned	Usually not assigned
Typical Resources	Web Servers, Load Balancers, Bastion Hosts	Application Servers, Databases, Internal Services
Security	Accessible from the internet (with proper security rules)	Isolated from the internet

Public Subnet

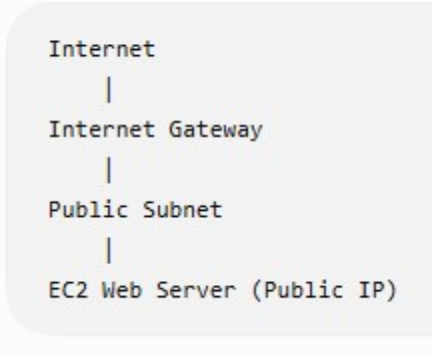
Features

- Has a route to an **Internet Gateway (IGW)**.
- Instances can have **Public IPs** or **Elastic IPs**.
- Can communicate directly with the internet.

Common Resources

- Web Servers
- Application Load Balancers
- Bastion Hosts
- NAT Gateways

Diagram



Private Subnet

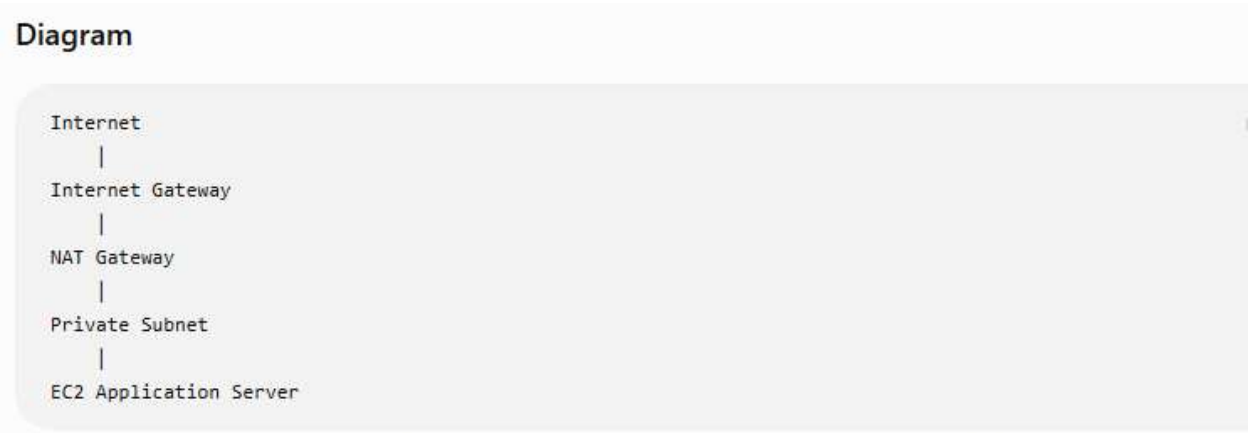
Features

- No direct route to the Internet Gateway.
- Instances normally have only private IP addresses.
- More secure because they are not directly reachable from the internet.

Common Resources

- Application Servers
- Amazon RDS Databases
- Internal APIs
- Backend Services

Diagram



The application server can **download software updates** through the NAT Gateway but **cannot receive inbound connections directly from the internet**.

Real-Time Architecture

Traffic Flow

1. User accesses the website.
2. Request reaches the Load Balancer in the Public Subnet.
3. Load Balancer forwards traffic to the Application Server in the Private Subnet.
4. Application Server communicates with the Database in the Private Subnet.

Why Use a NAT Gateway?

A Private Subnet cannot access the internet directly.

If the application server needs to:

- Install software updates
- Download packages
- Access AWS services over the internet

It sends outbound traffic through a **NAT Gateway** located in a Public Subnet.

Interview Questions

Q1: Can an EC2 instance in a Private Subnet access the internet?

Answer:

Yes, but only for **outbound** traffic through a **NAT Gateway** or **NAT Instance**. It cannot receive inbound connections directly from the internet.

Q2: Can a Database be placed in a Public Subnet?

Answer:

It is technically possible, but **not recommended**. Databases should normally be deployed in **Private Subnets** to improve security and reduce exposure to the internet.

Q3: Which resources are typically placed in a Public Subnet?

Answer:

- Web Servers
- Application Load Balancers
- Bastion Hosts
- NAT Gateways

Easy Memory Trick

Public Subnet	Private Subnet
Public = Internet	**Private = Internal**
Web Servers	Databases
Load Balancer	Application Server
Bastion Host	Backend Services

2-Minute Interview Summary

"A Public Subnet has a route to an Internet Gateway, allowing resources with public IP addresses to communicate directly with the internet. It is commonly used for web servers, load balancers, and bastion hosts. A Private Subnet has no direct route to the Internet Gateway and is used for application servers, databases, and internal services. If resources in a Private Subnet require outbound internet access, they use a NAT Gateway. This architecture improves security by exposing only the frontend components to the internet while keeping backend systems protected."

☆ Interview Tip

If asked, **"How do users access an application hosted in a private subnet?"**, answer:

Users do not access private subnet instances directly. They connect to a Load Balancer in the Public Subnet, which securely forwards requests to application servers in the Private Subnet.

What is a NAT Gateway? (AWS Interview Answer)

Definition

A **NAT Gateway (Network Address Translation Gateway)** is an AWS managed service that allows **EC2 instances in a private subnet to access the internet for outbound traffic**, while preventing the internet from initiating inbound connections to those instances.

Interview Answer (2 Minutes)

"A NAT Gateway is an AWS managed service that enables EC2 instances in a private subnet to access the internet for outbound communication, such as downloading software updates or accessing AWS services, without exposing those instances to inbound internet traffic. The NAT Gateway is deployed in a public subnet, is associated with an Elastic IP address, and the private subnet's route table sends internet-bound traffic to the NAT Gateway."

Why Do We Need a NAT Gateway?

Imagine you have an application server in a **Private Subnet**.

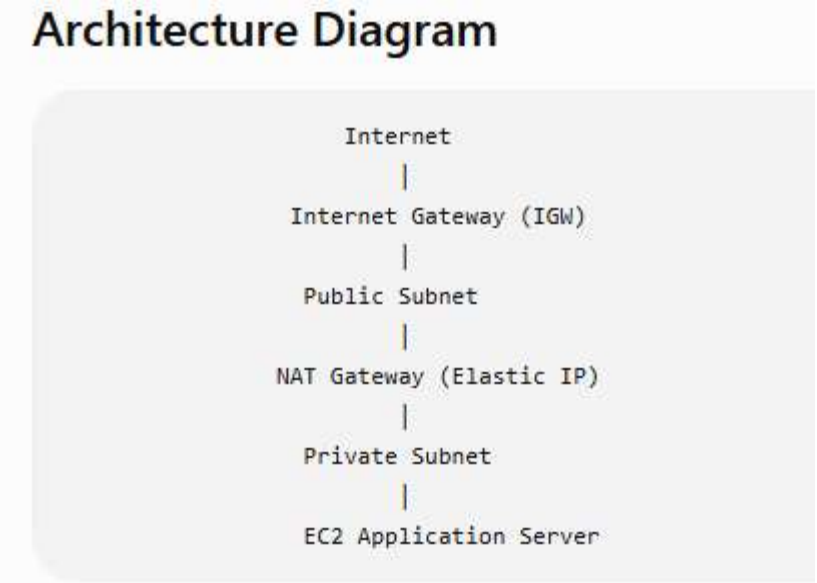
The server needs to:

- Download application updates
- Install packages using `yum` or `apt`
- Access AWS services
- Connect to external APIs

Since the server is in a private subnet, it **cannot access the internet directly**.

A **NAT Gateway** solves this problem.

Architecture Diagram



Traffic Flow

1. EC2 in the private subnet sends a request to the internet.
2. The route table forwards the traffic to the NAT Gateway.
3. The NAT Gateway sends the request through the Internet Gateway.
4. The response returns through the NAT Gateway to the EC2 instance.
5. **No inbound internet connection can directly reach the EC2 instance.**

How to Configure a NAT Gateway

Step 1

Create a **Public Subnet**.

Step 2

Attach an **Internet Gateway (IGW)** to the VPC.

Step 3

Allocate an **Elastic IP**.

Step 4

Create the **NAT Gateway** in the Public Subnet.

Step 5

Update the **Private Subnet Route Table**:

Destination	Target
0.0.0.0/0	NAT Gateway

Real-Time Example

Suppose your company has:

- **Web Server** → Public Subnet
- **Application Server** → Private Subnet
- **Database** → Private Subnet

The application server needs to:

- Download Java updates
- Install security patches
- Connect to a payment gateway API

The Application Server uses the **NAT Gateway** for outbound internet access while remaining inaccessible from the internet.

NAT Gateway vs Internet Gateway

Feature	NAT Gateway	Internet Gateway
Purpose	Outbound internet access for private subnet	Internet access for public subnet
Location	Public Subnet	Attached to the VPC
Requires Elastic IP	Yes	No
Inbound Internet Access	No	Yes (for resources with public IPs)
Used By	Private subnet instances	Public subnet instances

Common Interview Questions

Q1: Can a NAT Gateway receive inbound internet traffic?

Answer:

No. A NAT Gateway allows **only outbound** internet access. It does not accept unsolicited inbound connections from the internet.

Q2: Where should a NAT Gateway be deployed?

Answer:

A NAT Gateway must be created in a **Public Subnet** and must have an **Elastic IP**.

Q3: Does a NAT Gateway require an Internet Gateway?

Answer:

Yes. The NAT Gateway uses the Internet Gateway to send outbound traffic to the internet.

Q4: What happens if the NAT Gateway fails?

Answer:

A NAT Gateway is **highly available within a single Availability Zone (AZ)**. For high availability across multiple AZs, AWS recommends deploying **one NAT Gateway in each AZ** and configuring each private subnet to use the NAT Gateway in its own AZ.

Easy Memory Trick

- **Public Subnet** → Uses **Internet Gateway**
- **Private Subnet** → Uses **NAT Gateway**
- **IGW** → Inbound + Outbound (for public resources)
- **NAT Gateway** → Outbound only (for private resources)

2-Minute Interview Summary

"A NAT Gateway is an AWS managed service that enables EC2 instances in a private subnet to access the internet for outbound traffic while preventing inbound internet access. It is deployed in a public subnet, associated with an Elastic IP, and works with an Internet Gateway. The private subnet's route table directs internet-bound traffic to the NAT Gateway. It is commonly used for downloading updates, accessing external APIs, and reaching AWS services securely without exposing private instances to the internet."

☆ Interview Tip

A very common follow-up question is:

Q: Why not place the application server in a Public Subnet instead of using a NAT Gateway?

Answer:

Placing the application server in a **Private Subnet** reduces its exposure to the internet, improving security. The **NAT Gateway** allows the server to initiate outbound connections (such as downloading updates or calling external APIs) without allowing direct inbound internet access. This follows AWS security best practices.

Traffic Flow

1. EC2 in the private subnet sends a request to the internet.
2. The route table forwards the traffic to the NAT Gateway.
3. The NAT Gateway sends the request through the Internet Gateway.
4. The response returns through the NAT Gateway to the EC2 instance.
5. **No inbound internet connection can directly reach the EC2 instance.**

How to Configure a NAT Gateway

Step 1

Create a **Public Subnet**.

Step 2

Attach an **Internet Gateway (IGW)** to the VPC.

Step 3

Allocate an **Elastic IP**.

Step 4

Create the **NAT Gateway** in the Public Subnet.

Step 5

Update the **Private Subnet Route Table**:

Destination	Target
0.0.0.0/0	NAT Gateway

Real-Time Example

Suppose your company has:

- **Web Server** → Public Subnet
- **Application Server** → Private Subnet
- **Database** → Private Subnet

The application server needs to:

- Download Java updates
- Install security patches
- Connect to a payment gateway API

The Application Server uses the **NAT Gateway** for outbound internet access while remaining inaccessible from the internet.

NAT Gateway vs Internet Gateway

Feature	NAT Gateway	Internet Gateway
Purpose	Outbound internet access for private subnet	Internet access for public subnet
Location	Public Subnet	Attached to the VPC
Requires Elastic IP	Yes	No
Inbound Internet Access	No	Yes (for resources with public IPs)

Feature	NAT Gateway	Internet Gateway
Used By	Private subnet instances	Public subnet instances

Common Interview Questions

Q1: Can a NAT Gateway receive inbound internet traffic?

Answer:

No. A NAT Gateway allows **only outbound** internet access. It does not accept unsolicited inbound connections from the internet.

Q2: Where should a NAT Gateway be deployed?

Answer:

A NAT Gateway must be created in a **Public Subnet** and must have an **Elastic IP**.

Q3: Does a NAT Gateway require an Internet Gateway?

Answer:

Yes. The NAT Gateway uses the Internet Gateway to send outbound traffic to the internet.

Q4: What happens if the NAT Gateway fails?

Answer:

A NAT Gateway is **highly available within a single Availability Zone (AZ)**. For high availability across multiple AZs, AWS recommends deploying **one NAT Gateway in each AZ** and configuring each private subnet to use the NAT Gateway in its own AZ.

Easy Memory Trick

- **Public Subnet** → Uses **Internet Gateway**
- **Private Subnet** → Uses **NAT Gateway**
- **IGW** → Inbound + Outbound (for public resources)
- **NAT Gateway** → Outbound only (for private resources)

2-Minute Interview Summary

"A NAT Gateway is an AWS managed service that enables EC2 instances in a private subnet to access the internet for outbound traffic while preventing inbound internet access. It is deployed in a public subnet, associated with an Elastic IP, and works with an Internet Gateway. The private subnet's route table directs internet-bound traffic to the NAT Gateway. It is commonly used for downloading updates, accessing external APIs, and reaching AWS services securely without exposing private instances to the internet."

☆ Interview Tip

A very common follow-up question is:

Q: Why not place the application server in a Public Subnet instead of using a NAT Gateway?

Answer:

Placing the application server in a **Private Subnet** reduces its exposure to the internet, improving security. The **NAT Gateway** allows the server to initiate outbound connections (such as downloading updates or calling external APIs) without allowing direct inbound internet access. This follows AWS security best practices.

What is an Application Load Balancer (ALB)? (AWS Interview Answer)

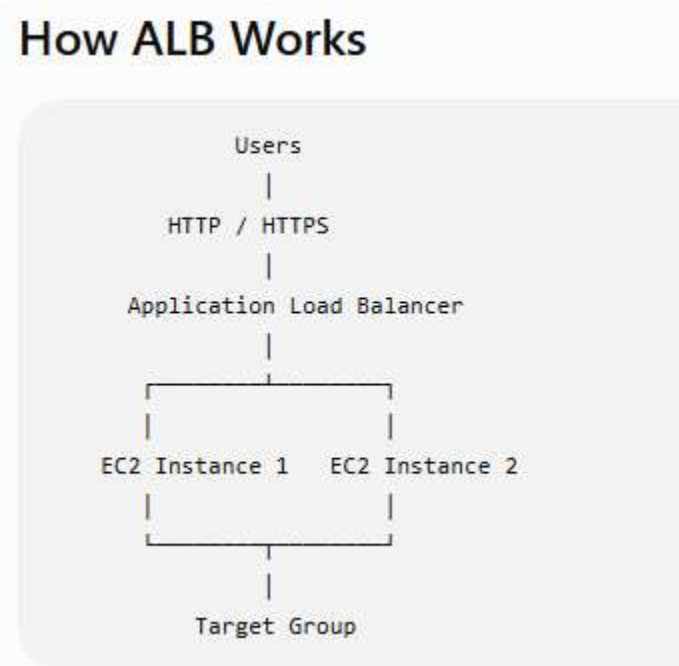
Definition

An **Application Load Balancer (ALB)** is an AWS **Layer 7 (Application Layer)** load balancer that distributes incoming **HTTP and HTTPS** traffic across multiple targets, such as EC2 instances, containers, IP addresses, and Lambda functions.

It improves **availability**, **scalability**, and **fault tolerance** by routing requests to healthy targets.

Interview Answer (2 Minutes)

"An Application Load Balancer (ALB) is a Layer 7 load balancer in AWS that distributes HTTP and HTTPS requests across multiple targets, such as EC2 instances, containers, or Lambda functions. It performs health checks to ensure traffic is sent only to healthy targets. ALB supports advanced routing features such as host-based routing, path-based routing, and SSL/TLS termination. It is commonly used for web applications to improve availability, scalability, and performance."



Flow

1. User sends an HTTP/HTTPS request.
2. The request reaches the **Application Load Balancer**.
3. ALB checks which targets are healthy.
4. ALB forwards the request to one of the healthy EC2 instances.
5. The EC2 instance processes the request and returns the response.

Key Features

1. Layer 7 Load Balancing

- Works with **HTTP** and **HTTPS** traffic.
- Makes routing decisions based on request content.

---

2. Health Checks

- Continuously monitors registered targets.
- Sends traffic only to healthy instances.

Example:

- EC2-1 → Healthy ✅
- EC2-2 → Unhealthy ❌

Traffic goes only to EC2-1 until EC2-2 recovers.

---

3. Path-Based Routing

Routes requests based on the URL path.

Example:

URL	Destination
`/login`	Login Server
`/images`	Image Server
`/api`	API Server

---

4. Host-Based Routing

Routes traffic based on the domain name.

Example:

Domain	Target
`app.example.com`	Application Server
`admin.example.com`	Admin Server

---

5. SSL/TLS Termination

- ALB can handle HTTPS encryption.
- EC2 instances can receive unencrypted HTTP traffic from the ALB if desired.
- Simplifies certificate management using AWS Certificate Manager (ACM).

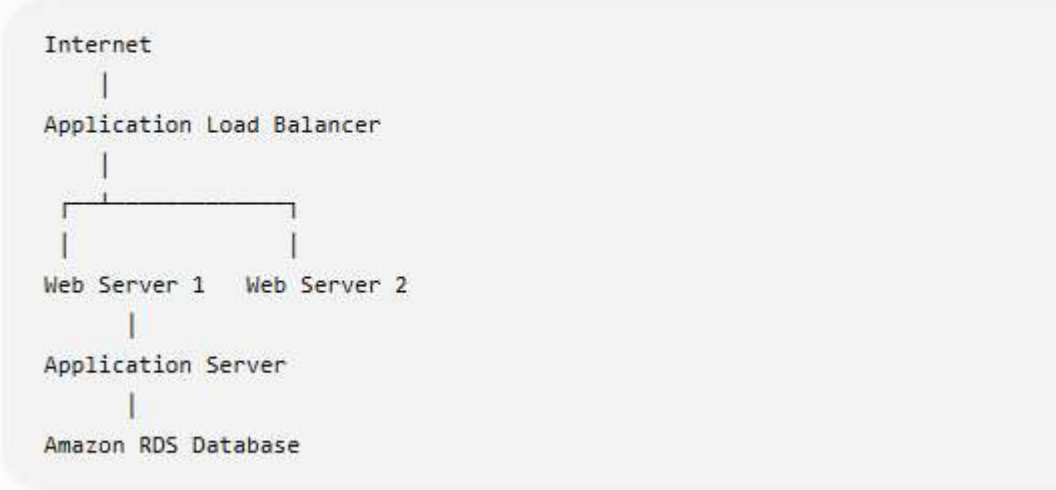
---

6. High Availability

- Can distribute traffic across **multiple Availability Zones (AZs)**.
- Improves fault tolerance.

Real-Time Example

An e-commerce website:



If **Web Server 1** fails, the ALB automatically routes traffic to **Web Server 2**.

Common Interview Questions

Q1: Why do we use an Application Load Balancer?

Answer:

- Distributes traffic across multiple servers.
- Improves availability and fault tolerance.
- Prevents a single server from becoming overloaded.
- Automatically routes traffic only to healthy targets.

Q2: Which OSI layer does an ALB operate on?

Answer:

Layer 7 (Application Layer).

Q3: What protocols does ALB support?

Answer:

- HTTP
- HTTPS

Q4: Can ALB route requests based on URLs?

Answer:

Yes. ALB supports **path-based routing** (for example, `/api`, `/images`) and **host-based routing** (for example, `app.example.com`, `admin.example.com`).

Application Load Balancer vs Network Load Balancer

Feature	Application Load Balancer (ALB)	Network Load Balancer (NLB)
OSI Layer	Layer 7	Layer 4
Protocols	HTTP, HTTPS	TCP, UDP, TLS
Routing	Path-based and host-based	IP address and port
Best For	Web applications	High-performance or low-latency applications

---

Easy Memory Trick

- **ALB = Application = Layer 7 = HTTP/HTTPS**
- **NLB = Network = Layer 4 = TCP/UDP**

2-Minute Interview Summary

"An Application Load Balancer is a Layer 7 load balancer that distributes HTTP and HTTPS requests across multiple targets, such as EC2 instances, containers, and Lambda functions. It performs health checks to send traffic only to healthy targets and supports advanced routing features like path-based routing, host-based routing, and SSL/TLS termination. ALBs are commonly used in highly available web applications and can distribute traffic across multiple Availability Zones for improved scalability and fault tolerance."

☆ Interview Tip

A common follow-up question is:

Q: Can an ALB send traffic directly to an EC2 instance in a private subnet?

Answer:

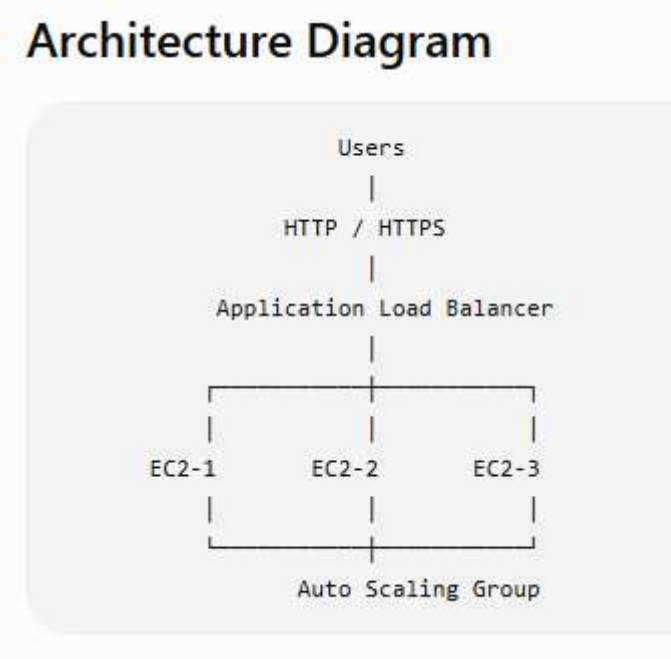
Yes. The ALB itself is typically deployed in **public subnets** so it can receive internet traffic, while its **target group** can contain EC2 instances in **private subnets**. This is a common and recommended architecture because it keeps the application servers protected from direct internet access.

How do Application Load Balancer (ALB) and Auto Scaling work together?

This is one of the **most frequently asked AWS interview questions** because ALB and Auto Scaling are commonly used together in production environments.

Interview Answer (2 Minutes)

"Application Load Balancer (ALB) and Auto Scaling work together to provide a highly available and scalable application. The ALB distributes incoming HTTP/HTTPS requests across multiple EC2 instances, while Auto Scaling automatically adds or removes EC2 instances based on demand. New instances are automatically registered with the ALB's target group, and unhealthy instances are removed. This ensures high availability, fault tolerance, and cost optimization."



How They Work Together

Step 1: User Requests

Users send requests to the **Application Load Balancer (ALB)**.

↓

Step 2: ALB Distributes Traffic

The ALB distributes requests evenly across all **healthy EC2 instances**.

↓

Step 3: Traffic Increases

Suppose website traffic suddenly increases from **100 users to 10,000 users**.

↓

Step 4: Auto Scaling Launches New Instances

The Auto Scaling Group monitors metrics like **CPU utilization**.

Example policy:

- CPU > 70% → Add 2 EC2 instances
- CPU < 30% → Remove 1 EC2 instance

Step 5: ALB Registers New Instances

As soon as the new EC2 instances become healthy, they are automatically added to the **ALB Target Group**.

The ALB immediately starts sending traffic to them.

Health Checks

The ALB continuously performs health checks.

Example:

```
EC2-1   ✅ Healthy
EC2-2   ❌ Unhealthy
EC2-3   ✅ Healthy
```

The ALB **stops sending traffic to EC2-2** until it becomes healthy again.

If Auto Scaling detects the unhealthy instance, it can terminate it and launch a replacement.

Real-Time Example

Imagine an online shopping website during a festival sale.

Normal Day

```
Users
   |
  ALB
   |
EC2-1
EC2-2
```

Two EC2 instances are enough.

Festival Sale

Traffic increases dramatically.

```
Users
   |
  ALB
   |
EC2-1
EC2-2
EC2-3
EC2-4
EC2-5
```

Auto Scaling launches three additional instances.

The ALB automatically distributes traffic across all five servers.

After the Sale

Traffic drops.

Auto Scaling terminates the extra EC2 instances.

The ALB removes them from the target group.

This reduces AWS costs.

Benefits

- ✅ High Availability
- ✅ Automatic Scaling
- ✅ Load Distribution
- ✅ Fault Tolerance
- ✅ Better Performance
- ✅ Cost Optimization

Interview Questions

Q1: Does ALB create EC2 instances?

Answer:

No. ALB only distributes traffic. **Auto Scaling** is responsible for launching and terminating EC2 instances.

Q2: How does ALB know about new EC2 instances?

Answer:

The Auto Scaling Group registers new EC2 instances with the **ALB Target Group**. After they pass health checks, the ALB starts routing traffic to them.

Q3: What happens if an EC2 instance becomes unhealthy?

Answer:

The ALB stops routing traffic to the unhealthy instance. If the instance belongs to an Auto Scaling Group, Auto Scaling can terminate it and launch a new one automatically.

ALB vs Auto Scaling

Application Load Balancer (ALB)	Auto Scaling
Distributes incoming traffic	Adds or removes EC2 instances
Performs health checks	Monitors metrics like CPU and memory (through CloudWatch)
Routes traffic to healthy targets	Replaces unhealthy instances
Improves availability	Improves scalability and cost efficiency

---

Easy Memory Trick

- **ALB = Traffic Manager**
- **Auto Scaling = Server Manager**

ALB decides where requests go.

Auto Scaling decides how many servers are needed.

2-Minute Interview Summary

"Application Load Balancer and Auto Scaling work together to build highly available and scalable applications. The ALB receives HTTP and HTTPS requests and distributes them across healthy EC2 instances. The Auto Scaling Group monitors demand using metrics such as CPU utilization and automatically launches or terminates EC2 instances based on scaling policies. New instances are automatically registered with the ALB target group after passing health checks, while unhealthy instances are removed and replaced. Together, ALB and Auto Scaling improve performance, fault tolerance, high availability, and cost optimization."

☆ Interview Tip

If the interviewer asks:

"Can Auto Scaling work without an ALB?"

Answer:

Yes. Auto Scaling can launch and terminate EC2 instances on its own. However, in most production web applications, it is paired with an **Application Load Balancer** so that incoming traffic is automatically distributed across all healthy instances. This combination provides both **scalability** and **high availability**.

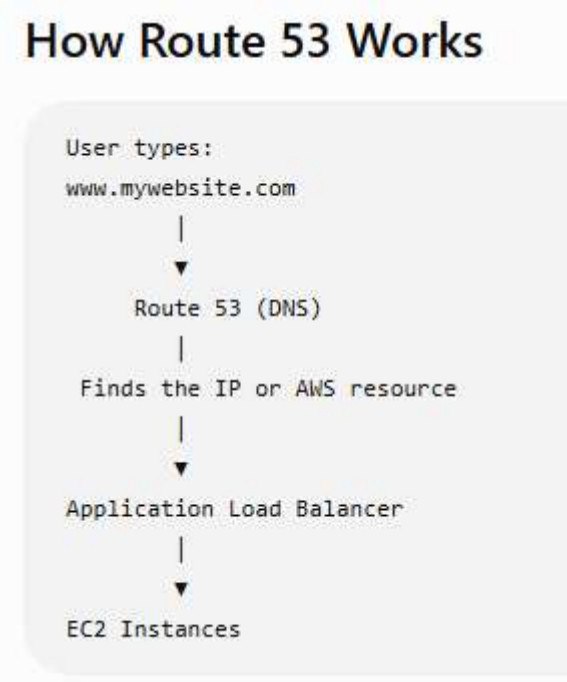
What is Amazon Route 53? (AWS Interview Answer)

Definition

Amazon Route 53 is a **highly available and scalable DNS (Domain Name System) and domain registration service** provided by AWS. It translates **domain names** (such as `www.example.com`) into **IP addresses** so users can access websites and applications.

Interview Answer (2 Minutes)

"Amazon Route 53 is AWS's highly available and scalable DNS service. It is used to register domain names, manage DNS records, and route user requests to AWS resources such as Application Load Balancers, EC2 instances, CloudFront distributions, or S3 static websites. Route 53 also supports health checks and multiple routing policies, helping improve application availability, performance, and reliability."



Flow

1. The user enters **www.mywebsite.com** in the browser.
2. Route 53 receives the DNS request.
3. Route 53 resolves the domain name to the appropriate AWS resource.
4. The request is forwarded to the Application Load Balancer or EC2 instance.
5. The website is displayed to the user.

Key Features

1. DNS Service

- Converts domain names into IP addresses.
- Makes websites easy to access without remembering IP addresses.

---

2. Domain Registration

- You can register domain names like:
  - `example.com`
  - `mycompany.in`

---

3. Health Checks

- Continuously monitors application endpoints.
- Stops routing traffic to unhealthy resources.

---

4. Traffic Routing

Route 53 can route traffic based on different policies.

Route 53 Routing Policies

1. Simple Routing

- Routes traffic to a single resource.

Example:

```
www.example.com → EC2 Instance
```

---

2. Weighted Routing

- Splits traffic based on assigned percentages.

Example:

- Server A → 80%
- Server B → 20%

Useful for gradual deployments or A/B testing.

---

3. Latency-Based Routing

- Routes users to the AWS Region with the lowest network latency.

Example:

- User in India → Mumbai Region
- User in Europe → Frankfurt Region

---

4. Failover Routing

- Sends traffic to a backup resource if the primary resource becomes unhealthy.

Example:

```
Primary ALB
     |
Healthy? Yes → Use Primary
Healthy? No  → Switch to Backup
```

---

5. Geolocation Routing

- Routes users based on their geographic location.

Example:

- India → Mumbai Server
- USA → Virginia Server

Real-Time Example

Suppose your company hosts a shopping website.

```
Users
   |
Route 53
   |
Application Load Balancer
   |
Auto Scaling Group
   |
EC2 Instances
```

- Users access `www.shop.com`.
- Route 53 resolves the domain.
- The request goes to the ALB.
- The ALB distributes traffic to healthy EC2 instances.

Common Interview Questions

Q1: Is Route 53 a DNS service?

Answer:

Yes. Route 53 is AWS's managed DNS service.

Q2: Why is it called Route 53?

Answer:

It is named after **port 53**, the standard TCP/UDP port used by DNS.

Q3: Can Route 53 perform health checks?

Answer:

Yes. Route 53 can monitor endpoints and automatically route traffic away from unhealthy resources.

Q4: Can Route 53 route traffic to an Application Load Balancer?

Answer:

Yes. Route 53 commonly routes traffic to an ALB, CloudFront distribution, EC2 instance, or S3 static website.

Easy Memory Trick

- **Route 53 = DNS + Domain Registration + Traffic Routing**
- **53 = DNS Port Number**

2-Minute Interview Summary

"Amazon Route 53 is AWS's highly available and scalable DNS service. It translates domain names into IP addresses and routes user requests to AWS resources such as Application Load Balancers, EC2 instances, CloudFront distributions, or S3 websites. It also provides domain registration, health checks, and advanced routing policies such as Simple, Weighted, Latency-Based, Failover, and Geolocation routing. Route 53 improves application availability, performance, and reliability by directing users to the most appropriate and healthy resources."

☆ Interview Tip

A common interview question is:

Q: How do Route 53, ALB, and Auto Scaling work together?

Answer:

```
User
   |
Route 53 (DNS)
   |
Application Load Balancer
   |
Auto Scaling Group
   |
EC2 Instances
```

- **Route 53** resolves the domain name and directs the user to the **Application Load Balancer**.
- The **ALB** distributes requests across healthy EC2 instances.
- **Auto Scaling** automatically adds or removes EC2 instances based on traffic demand, ensuring high availability and scalability.

Interview Question: What Route 53 routing policies have you used?

If you have real AWS project experience

Answer honestly based on what you've actually used. For example:

"In my projects, I have mainly used Simple Routing for basic applications and Failover Routing with health checks for high availability. I have also configured Weighted Routing for gradual application deployments and testing. I understand other routing policies such as Latency-Based and Geolocation Routing, although I haven't implemented them in a production environment."

If you're a fresher or transitioning into AWS/Linux

Since you've mentioned you're preparing for AWS/Linux interviews, it's better **not to claim experience you don't have**. A good answer is:

"During my AWS practice and labs, I worked with Simple Routing and Failover Routing. I understand how Weighted, Latency-Based, and Geolocation Routing work, and I know when to use them. Although I haven't implemented all of them in a production environment yet, I'm confident I can configure them based on project requirements."

This answer is truthful and shows both knowledge and willingness to learn.

Common Route 53 Routing Policies

Routing Policy	When to Use	Example
Simple Routing	One resource	One website hosted on a single ALB
Weighted Routing	Split traffic	80% to Version 1, 20% to Version 2
Latency-Based Routing	Lowest latency	Route Indian users to Mumbai, US users to Virginia
Failover Routing	Disaster recovery	Primary server fails, traffic moves to backup
Geolocation Routing	Route by user location	Different content for India and Europe
Geoproximity Routing	Route based on location and traffic bias	Direct users to the nearest AWS Region with optional traffic shifting
Multi-Value Answer Routing	Improve availability	Return multiple healthy IP addresses

---

Real-Time Example

Suppose you have an e-commerce application:

```
Users
   |
Route 53
   |
Application Load Balancer
   |
Auto Scaling Group
```

You could use:

- **Simple Routing** for a small application.
- **Weighted Routing** when deploying a new version gradually.
- **Failover Routing** to switch to a backup site if the primary site fails.
- **Latency-Based Routing** if your application is deployed in multiple AWS Regions.

Interview Tip

If the interviewer asks:

"Which routing policy is most commonly used?"

A good answer is:

"For single-region applications, Simple Routing is very common. In production environments that require high availability, Failover Routing is widely used. For global applications, Latency-Based Routing is commonly used to provide the best user experience by directing users to the nearest healthy AWS Region."

How do you monitor your AWS infrastructure? (AWS Interview Answer)

Interview Answer (2 Minutes)

"I monitor AWS infrastructure using Amazon CloudWatch, AWS CloudTrail, AWS Config, and Amazon SNS. CloudWatch is used to collect metrics, logs, and create alarms for resources like EC2, RDS, and Application Load Balancers. CloudTrail records all API calls made in the AWS account for auditing and security. AWS Config tracks configuration changes and helps ensure compliance. I use Amazon SNS to send email or SMS notifications whenever CloudWatch alarms are triggered."

AWS Monitoring Services

AWS Service	Purpose
Amazon CloudWatch	Monitors metrics, logs, and creates alarms
AWS CloudTrail	Records AWS API calls for auditing
AWS Config	Tracks resource configuration changes
Amazon SNS	Sends notifications (Email/SMS)

AWS Service	Purpose
AWS Health Dashboard	Shows AWS service health and maintenance events

---

1. Amazon CloudWatch

CloudWatch is the primary monitoring service in AWS.

It monitors:

- EC2 CPU Utilization
- Memory (with CloudWatch Agent)
- Disk Usage (with CloudWatch Agent)
- Network In/Out
- Application Logs
- ALB Metrics
- RDS Metrics

Example:

If CPU utilization exceeds **80%**, CloudWatch triggers an alarm.

```
EC2 Instance
     |
CloudWatch monitors CPU
     |
CPU > 80%
     |
CloudWatch Alarm
     |
SNS Email Notification
```

---

2. CloudWatch Alarms

Create alarms for:

- CPU Utilization
- Disk Space
- Memory Usage
- Network Traffic
- Status Check Failures

Example:

- CPU > 80% for 5 minutes
- Send email notification
- Trigger Auto Scaling policy

---

3. Amazon SNS (Simple Notification Service)

SNS sends alerts when alarms are triggered.

Notifications can be sent through:

- Email
- SMS
- HTTP/HTTPS endpoints
- AWS Lambda

Example:

Subject: High CPU Alert
CPU utilization on EC2 instance `i-1234567890` exceeded 80%.

---

4. AWS CloudTrail

CloudTrail records every AWS API call.

Examples:

- Who launched an EC2 instance?
- Who deleted an S3 bucket?
- Who modified an IAM policy?

Useful for:

- Security audits
- Troubleshooting
- Compliance

---

5. AWS Config

AWS Config tracks:

- Resource configurations
- Configuration history
- Compliance with rules

Example:
If someone makes an S3 bucket public, AWS Config can detect the change.

Real-Time Example

Suppose your web application runs on:

- EC2
- Application Load Balancer
- Auto Scaling
- RDS

Monitoring setup:

```
Users
   |
Application Load Balancer
   |
EC2 Instances
   |
CloudWatch
   |
CloudWatch Alarm
   |
SNS Email
```

If CPU exceeds 80%:

1. CloudWatch detects it.
2. CloudWatch Alarm is triggered.
3. SNS sends an email.
4. Auto Scaling can launch another EC2 instance (if configured).

Common Interview Questions

Q1: Which AWS service is mainly used for monitoring?

Answer:

Amazon CloudWatch.

Q2: Which AWS service records API calls?

Answer:

AWS CloudTrail.

Q3: How do you get notified when CPU usage is high?

Answer:

Create a **CloudWatch Alarm** and configure it to send notifications through **Amazon SNS**.

Q4: How do you monitor memory usage on EC2?

Answer:

Memory usage is **not collected by default**. Install and configure the **CloudWatch Agent** on the EC2 instance to send memory and disk metrics to CloudWatch.

Easy Memory Trick

- **CloudWatch** → Monitor resources
- **CloudTrail** → Audit API activity
- **AWS Config** → Track configuration changes
- **SNS** → Send alerts

2-Minute Interview Summary

"I monitor AWS infrastructure using Amazon CloudWatch to collect metrics, logs, and create alarms. I configure CloudWatch Alarms to monitor CPU, memory, disk, and network usage, and use Amazon SNS to send notifications when thresholds are exceeded. AWS CloudTrail records all API calls for auditing and security, while AWS Config tracks resource configuration changes and compliance. Together, these services help ensure the infrastructure remains available, secure, and compliant."

☆ Interview Tip

A common follow-up question is:

Q: What metrics do you monitor for an EC2 instance?

Answer:

- CPU Utilization
- Memory Usage (using CloudWatch Agent)
- Disk Usage (using CloudWatch Agent)
- Network In/Out
- Status Check Failures
- Disk Read/Write Operations

These are the key metrics interviewers commonly expect you to mention.

Interview Question: What metrics do you monitor in AWS?

Interview Answer (2 Minutes)

"I monitor infrastructure health, performance, and availability using Amazon CloudWatch. For EC2 instances, I monitor CPU utilization, memory usage, disk usage, network traffic, and status checks. For Application Load Balancers, I monitor request count, target response time, and HTTP error codes. For Auto Scaling, I monitor the number of instances and scaling activities. For RDS, I monitor CPU, storage, connections, and free memory. I also configure CloudWatch alarms to notify the operations team through Amazon SNS when thresholds are exceeded."

---

1. EC2 Metrics

Metric	Purpose	Example Threshold
CPU Utilization	Monitor CPU load	Alert if > 80%
Memory Usage *(CloudWatch Agent)*	Monitor RAM usage	Alert if > 80%
Disk Usage *(CloudWatch Agent)*	Monitor disk space	Alert if > 85%
Network In/Out	Monitor incoming/outgoing traffic	Detect unusual spikes
Status Check Failed	Detect hardware or OS issues	Alert immediately
Disk Read/Write Ops	Monitor storage I/O	Identify storage bottlenecks

Note: Memory and disk metrics require the **CloudWatch Agent** because they are not collected by default.

---

2. Application Load Balancer (ALB) Metrics

Metric	Purpose
Request Count	Number of incoming requests
Target Response Time	Time taken by targets to respond
HTTP 4XX Errors	Client-side errors
HTTP 5XX Errors	Server-side errors
Healthy Host Count	Number of healthy targets
Unhealthy Host Count	Number of unhealthy targets

---

3. Auto Scaling Metrics

Metric	Purpose
Group Desired Capacity	Desired number of instances
Group InService Instances	Running instances
Scaling Activities	Launch/terminate events

---

4. Amazon RDS Metrics

Metric	Purpose
CPU Utilization	Database CPU load
Free Storage Space	Available storage
Database Connections	Number of active connections
Freeable Memory	Available RAM
Read/Write IOPS	Database disk performance

---

5. Amazon S3 Metrics

Metric	Purpose
Bucket Size	Total storage used
Number of Objects	Total files stored
Request Metrics	GET, PUT, DELETE requests

Real-Time Example

Suppose your application is receiving heavy traffic.

```
Users
   |
Application Load Balancer
   |
EC2 Instance
   |
CloudWatch
```

Monitoring shows:

- CPU Utilization = **92%**
- Memory Usage = **88%**
- ALB Target Response Time increasing
- HTTP 5XX errors increasing

Action:

- CloudWatch Alarm triggers.
- Amazon SNS sends an alert.
- Auto Scaling launches additional EC2 instances (if configured).

Common Interview Questions

Q1: Which EC2 metrics are available by default?

Answer:

- CPU Utilization
- Network In
- Network Out

- Disk Read Operations
- Disk Write Operations
- Status Check Failed

Q2: Which metrics require the CloudWatch Agent?

Answer:

- Memory Usage
- Disk Usage
- File System Utilization
- Swap Usage (if applicable)

Q3: What is the most important EC2 metric?

Answer:

CPU Utilization is one of the most commonly monitored metrics because it often drives Auto Scaling decisions and indicates whether an instance is overloaded.

Easy Memory Trick

Remember **CPU-MDN** for EC2:

- **C** → CPU Utilization
- **M** → Memory Usage
- **D** → Disk Usage
- **N** → Network In/Out

2-Minute Interview Summary

"I primarily monitor CPU utilization, memory usage, disk usage, network traffic, and EC2 status checks using Amazon CloudWatch. For Application Load Balancers, I monitor request count, response time, healthy host count, and HTTP 4XX/5XX errors. For Auto Scaling, I monitor desired capacity and scaling activities. For RDS, I monitor CPU, storage, memory, and database connections. I configure CloudWatch alarms and integrate them

with Amazon SNS to receive notifications and take timely action when thresholds are exceeded."

☆ Interview Tip

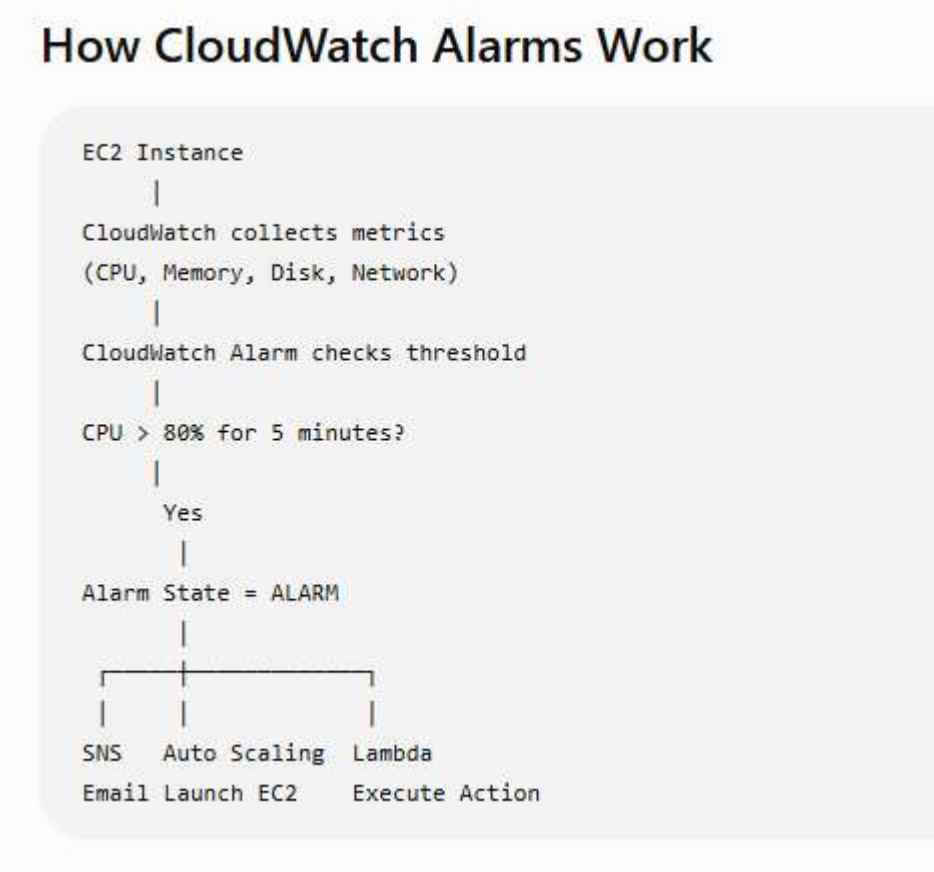
If you're interviewing for an **AWS/Linux Administrator** role, a concise answer that often impresses interviewers is:

"For EC2, I monitor CPU, memory, disk, network traffic, and status checks. For ALB, I monitor response time, request count, healthy hosts, and HTTP error codes. For RDS, I monitor CPU, storage, and connections. I use CloudWatch Alarms with SNS notifications, and I use Auto Scaling policies based on metrics like CPU utilization to maintain application performance and availability."

How do CloudWatch Alarms work? (AWS Interview Answer)

Interview Answer (2 Minutes)

"Amazon CloudWatch Alarms continuously monitor CloudWatch metrics against a threshold that I define. When the metric crosses the threshold for a specified evaluation period, the alarm changes its state from OK to ALARM and performs an action such as sending an Amazon SNS notification, triggering an Auto Scaling policy, or invoking an AWS Lambda function. When the metric returns to normal, the alarm changes back to the OK state."



Steps

Step 1: CloudWatch Collects Metrics

CloudWatch collects metrics from AWS resources.

Examples:

- CPU Utilization
- Network In/Out
- Disk I/O
- Status Checks

Note: Memory and disk utilization require the **CloudWatch Agent**.

Step 2: Create an Alarm

Define:

- Metric: CPU Utilization
- Threshold: 80%
- Evaluation Period: 5 minutes

Example:

```
CPU Utilization > 80%
Duration = 5 minutes
```

Step 3: Alarm Evaluates the Metric

CloudWatch continuously compares the current metric value with the configured threshold.

If:

```
CPU = 85%
```

The alarm enters the **ALARM** state.

Step 4: Perform an Action

CloudWatch Alarm can trigger:

- ✅ Amazon SNS (Email/SMS notification)
- ✅ Auto Scaling Policy
- ✅ AWS Lambda Function
- ✅ EC2 Recovery (for supported instances)

Alarm States

State	Meaning
OK	Metric is within the threshold
ALARM	Metric has crossed the threshold
INSUFFICIENT_DATA	Not enough data is available to determine the state

---

Real-Time Example

Suppose your web server is receiving heavy traffic.

Current metrics:

```
CPU = 92%
Memory = 85%
```

CloudWatch Alarm:

```
Threshold:
CPU > 80%

Evaluation:
5 minutes
```

Action:

```
CloudWatch Alarm
        |
        ▼
Amazon SNS
        |
Email to Admin
```

If Auto Scaling is configured:

```
CloudWatch Alarm
        |
        ▼
Auto Scaling
        |
Launch New EC2 Instance
```

Common Interview Questions

Q1: What actions can CloudWatch Alarms perform?

Answer:

- Send notifications using **Amazon SNS**
- Trigger **Auto Scaling**
- Invoke **AWS Lambda**
- Recover supported EC2 instances

Q2: What are the three CloudWatch Alarm states?

Answer:

1. **OK**
2. **ALARM**
3. **INSUFFICIENT_DATA**

Q3: Can CloudWatch Alarms monitor memory usage?

Answer:

Yes, but only after installing and configuring the **CloudWatch Agent** on the EC2 instance.

Q4: Can CloudWatch Alarms trigger Auto Scaling?

Answer:

Yes. A CloudWatch Alarm can trigger an Auto Scaling policy to launch or terminate EC2 instances based on metrics such as CPU utilization.

Real Production Example

Imagine an e-commerce website.

```
Users increase
      |
CPU reaches 90%
      |
CloudWatch Alarm
      |
Auto Scaling launches 2 new EC2 instances
      |
Application Load Balancer distributes traffic
      |
CPU returns to 45%
```

Result:

- Application remains available.
- Users experience better performance.

- No manual intervention is required.

Easy Memory Trick

CloudWatch Alarm = Monitor → Compare → Alert → Action

- **Monitor** → Collect metrics
- **Compare** → Check against threshold
- **Alert** → Change to ALARM state
- **Action** → SNS, Auto Scaling, Lambda, or EC2 recovery

2-Minute Interview Summary

"CloudWatch Alarms monitor AWS resource metrics such as CPU utilization, network traffic, and status checks. I configure a metric, define a threshold, and specify an evaluation period. If the metric exceeds the threshold, the alarm changes from the OK state to the ALARM state and can perform actions such as sending notifications through Amazon SNS, triggering Auto Scaling, invoking a Lambda function, or recovering an EC2 instance. When the metric returns to normal, the alarm automatically returns to the OK state."

☆ Interview Tip

A common scenario-based question is:

Q: Your EC2 CPU utilization remains above 80% for 5 minutes. What happens?

Answer:

1. CloudWatch detects that CPU utilization is above the configured threshold.
2. The alarm changes from **OK** to **ALARM**.
3. An **SNS notification** is sent to the operations team.
4. If an **Auto Scaling policy** is attached, it launches additional EC2 instances.
5. The **Application Load Balancer (ALB)** begins routing traffic to the new healthy instances once they pass health checks. This keeps the application responsive during high traffic.

Interview Question: A Linux server is running out of disk space. What commands do you use?

Interview Answer (2 Minutes)

"If a Linux server is running out of disk space, I first check overall disk usage using `df -h`. Then I identify which directories are consuming the most space using `du -sh` or `du -ah`. I check for large log files in `/var/log`, remove unnecessary files if appropriate, and identify deleted files that are still being held open using `lsof`. If needed, I clean package caches and old log files after verifying they are no longer required."

Step-by-Step Troubleshooting

1. Check Disk Usage

```
df -h
```

Purpose:

- Shows filesystem usage.
- Displays total, used, available space, and usage percentage.

Sample Output:

```
Filesystem      Size Used Avail Use%
/dev/xvda1       20G 19G   1G   95%
```

---

2. Find Which Directory Uses the Most Space

```
du -sh /*
```

Or for the current directory:

```
du -sh *
```

Purpose:

- Displays the size of each directory.
- Helps identify where disk space is being consumed.

---

3. Find Large Files

```
find / -type f -size +500M
```

Purpose:

- Finds files larger than 500 MB.

---

4. Check Log Files

```
cd /var/log
du -sh *
```

or

```
ls -lh /var/log
```

Purpose:

- Large log files are a common cause of disk space issues.

---

5. Check Deleted Files Still in Use

```
lsof | grep deleted
```

Purpose:

- Finds deleted files that are still open by running processes.
- Restarting the affected service may release the disk space.

---

6. Check Inode Usage

Sometimes disk space is available, but the server cannot create new files because **inodes** are exhausted.

```
df -i
```

---

7. Clean Package Cache (If Needed)

RHEL/CentOS

```
sudo yum clean all
```

or

```
sudo dnf clean all
```

Ubuntu

```
sudo apt clean
```

---

8. Remove Old Log Files (Carefully)

```
sudo journalctl --vacuum-time=7d
```

This removes **systemd journal logs** older than 7 days.

Always verify your organization's log retention policy before deleting logs.

Real-Time Scenario

Suppose the application team reports:

"The application is slow because the server disk is 100% full."

Troubleshooting Steps

```
df -h
```

Output:

```
/dev/xvda1   20G   20G   0G   100%
```

↓

```
du -sh /var/*
```

Output:

```
/var/log   12G
```

↓

```
ls -lh /var/log
```

Find:

```
application.log   10G
```

↓

Rotate or archive the log (according to your organization's procedure), then verify:

```
df -h
```

Disk usage returns to a healthy level.

Common Interview Questions

Q1: Which command checks disk usage?

```
df -h
```

Q2: Which command checks directory sizes?

```
du -sh *
```

Q3: Which command finds files larger than 1 GB?

```
find / -type f -size +1G
```

Q4: Which command checks inode usage?

```
df -i
```

Q5: How do you find deleted files still consuming disk space?

```
lsof | grep deleted
```

Easy Memory Trick

Remember:

- **df** → **D**isk **F**ilesystem usage
- **du** → **D**irectory **U**sage
- **find** → Locate large files
- **lsof** → Open files (including deleted ones)
- **df -i** → Inode usage

2-Minute Interview Summary

"When a Linux server runs out of disk space, I first check overall filesystem usage using `df -h`. Then I identify the directories consuming the most space using `du -sh`. I locate large files with the `find` command, inspect log files under `/var/log`, and check for deleted files that are still held open using `lsof | grep deleted`. I also verify inode usage with `df -i`. After identifying the root cause, I clean up or archive unnecessary data according to operational procedures and then confirm that disk usage has returned to normal."

☆ Interview Tip

A common follow-up question is:

Q: Which command do you run first when someone says, "The server is out of disk space"?

Answer:

```
df -h
```

This is the best first step because it quickly shows:

- Which filesystem is full
- Total space
- Used space
- Available space
- Percentage of disk usage

Linux Interview Question: Difference between `top`, `ps`, `free`, `df`, and `du`

This is one of the **most common Linux Admin interview questions**.

Interview Answer (2 Minutes)

"`top`, `ps`, `free`, `df`, and `du` are Linux monitoring commands, but each serves a different purpose. `top` provides a real-time view of running processes and system resource usage. `ps` displays information about currently running processes at a specific point in time. `free` shows memory and swap usage. `df` displays disk space usage for mounted file systems, while `du` shows disk usage for specific directories and files."

Comparison Table

Command	Purpose	Real-Time?	Common Usage
top	Monitor running processes and CPU/memory usage	✅ Yes	Troubleshooting high CPU or memory usage
ps	Display process information	❌ No	Find a specific process
free	Show RAM and swap usage	❌ No	Check available memory
df	Show filesystem disk usage	❌ No	Check free disk space
du	Show directory/file disk usage	❌ No	Find which directory is consuming space

---

1. `top`

Purpose

- Displays **real-time** system information.
- Shows:
  - CPU usage
  - Memory usage
  - Running processes
  - Process IDs (PID)

Command

```
top
```

Sample Output

```
PID   USER    %CPU    %MEM    COMMAND
1234  root     75      10     java
5678  ec2-user 20      5       nginx
```

Use Case

If the server is slow, run:

```
top
```

to identify processes consuming high CPU or memory.

---

2. `ps`

Purpose

Shows a snapshot of running processes.

Command

```
ps -ef
```

or

```
ps aux
```

Example

```
ps -ef | grep nginx
```

Use Case

Find whether a specific process (e.g., `nginx`) is running.

---

3. `free`

Purpose

Displays RAM and swap usage.

Command

```
free -h
```

Sample Output

```
Total   Used   Free
8G      4G     4G
```

Use Case

Check if the server has enough available memory.

---

4. `df`

Purpose

Shows disk space usage for mounted filesystems.

Command

```
df -h
```

Sample Output

```
Filesystem   Size   Used   Avail   Use%
/dev/xvda1    20G    15G      5G    75%
```

Use Case

Check whether a filesystem is running out of disk space.

---

5. `du`

Purpose

Shows disk usage of directories and files.

Command

```
du -sh /var/*
```

or

```
du -sh *
```

Sample Output

```
8G   /var/log
2G   /home
500M /tmp
```

Use Case

Identify which directory is consuming the most disk space.

Real-Time Example

Suppose users report that the server is slow.

Step 1: Check CPU usage

```
top
```

↓

High CPU used by Java.

Step 2: Check memory

```
free -h
```

↓

Memory usage is normal.

Step 3: Check disk space

```
df -h
```

↓

Filesystem is 95% full.

Step 4: Find which directory is using space

```
du -sh /var/*
```

↓

`/var/log` is consuming 10 GB.

You have identified the root cause.

Common Interview Questions

Q1: Which command shows real-time CPU usage?

Answer:

```
top
```

Q2: Which command shows memory usage?

Answer:

```
free -h
```

Q3: Which command checks filesystem disk space?

Answer:

```
df -h
```

Q4: Which command finds large directories?

Answer:

```
du -sh *
```

Q5: Which command lists running processes?

Answer:

```
ps -ef
```

or

```
ps aux
```

Easy Memory Trick

- **top → Top processes** (real-time)
- **ps → Process status**
- **free → Free memory**
- **df → Disk filesystem usage**
- **du → Directory usage**

2-Minute Interview Summary

"`top` is used to monitor processes and system performance in real time. `ps` provides a snapshot of running processes. `free` displays memory and swap usage. `df` reports disk space usage for mounted filesystems, while `du` identifies how much disk space specific directories or files are using. Together, these commands help Linux administrators troubleshoot CPU, memory, process, and disk-related issues efficiently."

☆ Interview Tip

If the interviewer asks:

"A Linux server is slow. Which commands would you use first?"

A good sequence is:

1. `top` → Check CPU and running processes.
2. `free -h` → Check memory usage.
3. `df -h` → Check disk space.
4. `du -sh /var/*` → Identify large directories if disk space is low.
5. `ps -ef` → Inspect specific processes.

Interview Question: How do you troubleshoot high CPU usage in Linux?

Interview Answer (2 Minutes)

"When I notice high CPU usage, I first identify the processes consuming the CPU using `top` or `htop`. Then I use `ps` to gather more details about the process. I verify whether the high CPU usage is expected, such as during backups or batch jobs, or caused by an application issue. I check system logs for errors, and if required, restart the affected service after confirming it is safe to do so. If the issue continues, I analyze the application logs and escalate it to the application team with my findings."

Step-by-Step Troubleshooting

Step 1: Check CPU Usage

```
top
```

or

```
htop
```

Look for:

- CPU utilization
- Process ID (PID)
- Process name
- User

Example:

```
PID    USER      %CPU   COMMAND
2345   root      95.5   java
```

Here, the **Java** process is consuming 95% CPU.

Step 2: Identify the Process

```
ps -fp 2345
```

or

```
ps -ef | grep java
```

This shows:

- Process owner
- Parent process
- Start time
- Command

Step 3: Check System Load

```
uptime
```

Example:

```
load average: 6.5, 7.2, 6.9
```

A consistently high load average may indicate CPU contention.

Step 4: Check Logs

System logs:

```
sudo journalctl -xe
```

or

```
tail -100 /var/log/messages
```

or (Ubuntu)

```
tail -100 /var/log/syslog
```

Look for:

- Application errors
- Out-of-memory events
- Hardware issues

Step 5: Check Resource Usage

Memory:

```
free -h
```

Disk:

```
df -h
```

A system with low memory may start swapping, making CPU usage appear high.

Step 6: Restart the Service (If Required)

Example:

```
sudo systemctl restart nginx
```

or

```
sudo systemctl restart httpd
```

Only restart services after confirming it won't impact users unexpectedly.

Step 7: Kill the Process (Last Resort)

Find the PID:

```
ps -ef | grep java
```

Terminate gracefully:

```
kill 2345
```

Force terminate if necessary:

```
kill -9 2345
```

Use `kill -9` only when the process does not respond to a normal termination.

Real-Time Scenario

Suppose users report that the application is slow.

Step 1

```
top
```

Output:

```
java  98% CPU
```

↓

Step 2

```
ps -fp <PID>
```

Confirm it's the application process.

↓

Step 3

Check application logs:

```
tail -100 /var/log/app.log
```

↓

Find repeated errors or excessive requests.

↓

Step 4

If required:

- Restart the service.
- Inform the application team.
- Continue monitoring CPU usage.

Common Interview Questions

Q1: Which command do you use first for high CPU usage?

`top`

Q2: Which command shows detailed process information?

`ps -ef`

Q3: How do you stop a process?

Graceful:

`kill PID`

Force:

`kill -9 PID`

Q4: How do you check system load?

`uptime`

Easy Memory Trick

Remember **TPULK**:

- **T** → `top` (Find high CPU process)
- **P** → `ps` (Process details)
- **U** → `uptime` (Check load average)
- **L** → Logs (`journalctl`, `/var/log`)

- **K** → `kill` or restart service (if required)

2-Minute Interview Summary

"To troubleshoot high CPU usage, I first use `top` or `htop` to identify the process consuming CPU. Then I use `ps` to get process details and `uptime` to check the system load average. I review system and application logs to identify the root cause, verify memory and disk usage, and determine whether the CPU usage is expected or abnormal. If necessary, I restart the affected service or terminate an unresponsive process after assessing the impact. If the issue is application-related, I collect evidence and coordinate with the application team."

☆ Interview Tip

If the interviewer asks:

"What if the CPU is high because of a Java application?"

A good answer is:

**"I first confirm the Java process is causing the high CPU usage using `top` and `ps`. Then I review the application logs to determine whether the issue is due to high traffic, a memory leak, inefficient code, or an application bug. If restarting the service is appropriate, I coordinate with the application team, restart the service if approved, and continue monitoring. If the problem persists, I escalate it with the collected logs and performance data."

Interview Question: What is your backup strategy?

Interview Answer (2 Minutes)

"My backup strategy follows the 3-2-1 principle whenever possible: keep three copies of data, store them on two different types of storage, and keep one copy offsite. In AWS, I use AWS Backup and Amazon EBS snapshots for EC2 volumes, Amazon RDS automated backups and snapshots for databases, and Amazon S3 Versioning with Lifecycle policies for object storage. I schedule backups based on business requirements, encrypt backup data, define retention policies, and regularly test restore procedures to ensure backups can be recovered successfully."

Backup Strategy by AWS Service

AWS Service	Backup Method
EC2	Amazon EBS Snapshots or AWS Backup
Amazon RDS	Automated Backups + Manual Snapshots
Amazon S3	Versioning, Cross-Region Replication (if required), Lifecycle Policies
Amazon EFS	AWS Backup
Amazon DynamoDB	Point-in-Time Recovery (PITR) or On-Demand Backups

---

EC2 Backup

For EC2 instances:

- Take **EBS Snapshots** of attached volumes.
- Use **AWS Backup** to automate backup schedules.
- Store backups according to the retention policy.

Example:

```
EC2 Instance
      |
EBS Volume
      |
EBS Snapshot
      |
Amazon S3 (managed by AWS)
```

RDS Backup

For Amazon RDS:

- Enable **Automated Backups**.
- Take **Manual Snapshots** before major changes or upgrades.

Benefits:

- Point-in-time recovery (within the retention period).
- Restore the database if data is lost or corrupted.

S3 Backup

For Amazon S3:

- Enable **Versioning** to recover deleted or overwritten objects.
- Use **Lifecycle Policies** to transition older data to lower-cost storage classes like S3 Glacier.
- Enable **Cross-Region Replication (CRR)** if disaster recovery across Regions is required.

Backup Best Practices

- Schedule automatic backups.
- Encrypt backups using AWS KMS.
- Define retention policies (for example, 30, 90, or 365 days).
- Restrict backup access using IAM.
- Test restore procedures regularly.

Real-Time Example

Suppose an e-commerce application consists of:

- EC2 Application Servers
- Amazon RDS Database
- Amazon S3 Bucket

Backup plan:

- **EC2** → Daily EBS Snapshots
- **RDS** → Automated Backups + Snapshot before upgrades
- **S3** → Versioning + Lifecycle Policy
- **AWS Backup** → Centralized backup scheduling and monitoring

If a server fails, restore from an EBS snapshot. If data is accidentally deleted from S3, recover a previous version. If the database is corrupted, restore it from an automated backup or snapshot.

Common Interview Questions

Q1: How do you back up an EC2 instance?

Answer:

I create **EBS snapshots** of the attached volumes or use **AWS Backup** to automate scheduled backups.

Q2: How do you back up an RDS database?

Answer:

I enable **Automated Backups** and take **manual snapshots** before maintenance or major changes.

Q3: How do you protect data in Amazon S3?

Answer:

I enable **Versioning**, configure **Lifecycle Policies**, and use **Cross-Region Replication** if disaster recovery requirements exist.

Q4: Why should backups be tested?

Answer:

A backup is valuable only if it can be restored successfully. Regular restore testing verifies that backups are complete, usable, and meet recovery objectives.

Easy Memory Trick

Remember **SVR**:

- **S** → Schedule backups
- **V** → Verify by testing restores
- **R** → Retention and Recovery planning

2-Minute Interview Summary

"My backup strategy uses AWS Backup and EBS snapshots for EC2, automated backups and snapshots for RDS, and Versioning with Lifecycle Policies for S3. I automate backups based on business requirements, encrypt backup data, define retention periods, and regularly test restoration procedures. This approach helps meet recovery objectives while ensuring data protection and business continuity."

☆ Interview Tip

A common follow-up question is:

Q: What is the difference between an EBS Snapshot and an AMI?

Answer:

- **EBS Snapshot**: Backs up the **data on an EBS volume**.
- **AMI (Amazon Machine Image)**: Creates a template that includes the operating system, configuration, and associated EBS snapshots, allowing you to launch new EC2 instances quickly.

Interview Question: How do you restore an EC2 instance?

Interview Answer (2 Minutes)

"The restoration method depends on the type of backup available. If I have an EBS snapshot, I create a new EBS volume from the snapshot and attach it to an EC2 instance, or I launch a new EC2 instance using a restored volume. If I have an AMI backup, I simply launch a new EC2 instance from the AMI. After restoration, I verify that the application, data, networking, and security settings are working correctly before returning the server to production."

Method 1: Restore Using an EBS Snapshot

This method is used when you have backed up the EC2 instance's EBS volume.

Steps

Step 1

Open **AWS Console → EC2 → Snapshots**

↓

Step 2

Select the required snapshot.

↓

Step 3

Choose **Create Volume**.

- Select the same **Availability Zone (AZ)** as the EC2 instance.

↓

Step 4

Attach the new EBS volume to the EC2 instance.

↓

Step 5

Mount the volume.

Example (Linux):

```
lsblk
sudo mkdir /restore
sudo mount /dev/xvdf1 /restore
```

Now the data is available under `/restore`.

Method 2: Restore Using an AMI

If you created an **AMI** of the EC2 instance:

Steps

1. Open **EC2 Console**
2. Select **AMIs**
3. Choose the required AMI
4. Click **Launch Instance**
5. Configure:
    - Instance type

    - VPC
    - Subnet
    - Security Group
    - Key Pair
6. Launch the instance.

This creates a new EC2 instance with the same operating system and configuration captured in the AMI.

Architecture

```
Original EC2
      |
EBS Snapshot / AMI
      |
Restore
      |
New EC2 Instance
```

Real-Time Scenario

Suppose an application server crashes because of disk corruption.

Backup available:

- Daily EBS Snapshots
- Weekly AMI

Recovery

1. Create a new volume from yesterday's snapshot.
2. Attach it to a replacement EC2 instance (or restore the original if appropriate).
3. Mount the volume.
4. Verify the application data.
5. Start the application service.
6. Test the application before making it available to users.

If the entire instance is lost, launch a new EC2 instance from the latest AMI.

Verification After Restore

After restoring:

- Verify the EC2 instance is running.
- Check the application status.

```
systemctl status nginx
```

or

```
systemctl status httpd
```

- Verify data is intact.
- Check logs.

```
tail -100 /var/log/messages
```

- Test application access.
- Confirm Security Groups, IAM role, and networking are correct.

Common Interview Questions

Q1: Can you restore an EC2 instance directly from an EBS snapshot?

Answer:

No. An EBS snapshot restores an **EBS volume**, not the EC2 instance itself. You create a new volume from the snapshot and attach it to an EC2 instance, or use it when launching a new instance.

Q2: Which is faster for full server recovery?

Answer:

An **AMI** is generally faster because it includes the operating system configuration and references the required EBS snapshots, allowing a new EC2 instance to be launched quickly.

Q3: What should you verify after restoration?

Answer:

- Application services are running.
- Data is complete.

- Security Groups and IAM role are correct.
- Networking is working.
- Users can access the application.

Easy Memory Trick

Remember **SAVV**:

- **S** → Snapshot or AMI
- **A** → Attach or Launch
- **V** → Verify services
- **V** → Validate application

2-Minute Interview Summary

"To restore an EC2 instance, I first identify the available backup. If I have an EBS snapshot, I create a new EBS volume from the snapshot and attach it to an EC2 instance. If I have an AMI, I launch a new EC2 instance directly from it. After the restoration, I verify the application, data, network configuration, security settings, and logs to ensure the server is functioning correctly before putting it back into production."

☆ Interview Tip

If the interviewer asks:

"What is the difference between restoring from an EBS Snapshot and restoring from an AMI?"

Answer:

EBS Snapshot	AMI
Restores an **EBS volume**	Launches a **new EC2 instance**
Used mainly to recover data	Used to recover the complete server (OS + configuration + attached snapshots)
Volume must be attached to an EC2 instance	Ready to boot as a new EC2 instance

Interview Question: How do you secure an AWS environment?

Interview Answer (2 Minutes)

"I secure an AWS environment using the principle of least privilege, network security, encryption, monitoring, and regular patching. I use IAM roles and policies to control access, enable MFA for privileged users, secure the network with Security Groups and Network ACLs, encrypt data using AWS KMS, monitor the environment with CloudWatch and CloudTrail, and regularly apply security updates to EC2 instances. I also enable logging, backup critical resources, and avoid storing sensitive credentials in the application code."

AWS Security Best Practices

1. IAM (Identity and Access Management)

- Follow the **Principle of Least Privilege**
- Create **IAM Roles** instead of storing access keys on EC2
- Enable **Multi-Factor Authentication (MFA)** for privileged users
- Rotate access keys regularly
- Avoid using the **root account** for daily activities

Example:

- EC2 accesses an S3 bucket using an **IAM Role** instead of Access Keys.

---

2. Network Security

Security Groups

- Allow only required ports.
- Example:
  - SSH (22) → Admin IP only
  - HTTP (80)
  - HTTPS (443)

Network ACLs

- Add subnet-level security.
- Use deny rules where appropriate.

---

3. Encryption

Encrypt data:

At Rest

- EBS Volumes
- S3 Buckets
- RDS Databases

Use **AWS KMS** to manage encryption keys.

In Transit

- HTTPS
- SSL/TLS certificates

---

4. Monitoring & Logging

Amazon CloudWatch

Monitor:

- CPU
- Memory (CloudWatch Agent)
- Disk
- Network
- Logs

AWS CloudTrail

Track:

- IAM changes
- EC2 launches
- S3 access
- API calls

---

5. Backup & Disaster Recovery

- EBS Snapshots
- RDS Automated Backups
- S3 Versioning

- AWS Backup

Regularly test restore procedures.

---

6. Patch Management

Keep EC2 instances updated.

Example:

Amazon Linux / RHEL

```
sudo yum update -y
```

Ubuntu

```
sudo apt update
sudo apt upgrade -y
```

---

7. Secrets Management

Never hardcode:

- AWS Access Keys
- Database passwords
- API Keys

Use:

- **AWS Secrets Manager**
- **AWS Systems Manager Parameter Store**

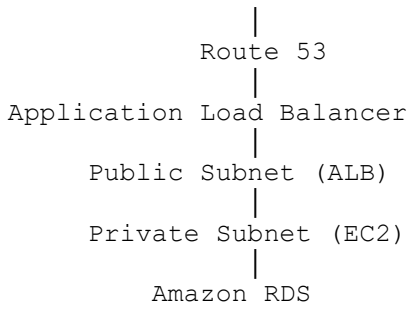
---

8. S3 Security

- Block Public Access unless required.
- Enable Bucket Versioning.
- Encrypt objects.
- Use Bucket Policies and IAM Policies.

Real-Time Architecture

```
Users
```



Security applied:

- Security Groups
- Network ACLs
- IAM Roles
- Encryption (KMS)
- CloudWatch
- CloudTrail
- AWS Backup

Common Interview Questions

Q1: How do you secure an EC2 instance?

Answer:

- Use Security Groups.
- Disable password-based SSH if possible and use SSH key pairs.
- Apply OS patches regularly.
- Assign an IAM Role instead of storing access keys.
- Encrypt EBS volumes.
- Monitor using CloudWatch.

Q2: How do you secure an S3 bucket?

Answer:

- Enable Block Public Access.
- Encrypt data.
- Enable Versioning.
- Apply Bucket Policies and IAM Policies.
- Enable logging if required.

Q3: Which AWS service records user activities?

Answer:

AWS CloudTrail records API calls and user activities.

Q4: Which AWS service manages encryption keys?

Answer:

AWS Key Management Service (AWS KMS).

Easy Memory Trick

Remember **INEMBPS**:

- **I** → IAM
- **N** → Network Security
- **E** → Encryption
- **M** → Monitoring
- **B** → Backup
- **P** → Patching
- **S** → Secrets Management

2-Minute Interview Summary

"To secure an AWS environment, I follow the principle of least privilege using IAM roles and policies, enable MFA for privileged users, secure network access with Security Groups and Network ACLs, encrypt data using AWS KMS, monitor infrastructure with CloudWatch and CloudTrail, regularly patch EC2 instances, protect secrets with AWS Secrets Manager or Systems Manager Parameter Store, and implement backup and disaster recovery using EBS snapshots, RDS backups, S3 Versioning, and AWS Backup. These practices help improve security, compliance, and operational reliability."

☆ Interview Tip

If the interviewer asks:

"What are the top five AWS security services?"

A strong answer is:

1. **IAM** – Identity and access management.
2. **Security Groups & Network ACLs** – Network-level security.
3. **AWS KMS** – Encryption key management.
4. **CloudTrail** – Audit logging and API activity.
5. **CloudWatch** – Monitoring and alerting.

These are the core AWS security services that are commonly discussed in AWS Administrator and DevOps interviews.

Interview Question: A production website is down. What are the first five things you check?

This is one of the **most common scenario-based interview questions** for Linux, AWS, and DevOps roles.

Interview Answer (2 Minutes)

"If a production website is down, I follow a structured troubleshooting approach. First, I check whether the EC2 instance is running and has passed its status checks. Second, I verify the Application Load Balancer health checks and target group status. Third, I check the Security Groups, Network ACLs, and Route Tables to ensure network connectivity. Fourth, I review the web server and application services, such as Nginx, Apache, or Tomcat, along with their logs. Finally, I check CloudWatch metrics, CloudWatch Alarms, and CloudTrail logs to identify any resource issues or recent configuration changes."

First 5 Things to Check

1. Check EC2 Instance Health

Verify:

- Is the EC2 instance running?
- Are the **2/2 Status Checks** passing?
- CPU, Memory, Disk usage

Linux commands:

```
systemctl status nginx
top
df -h
```

AWS Console:

- EC2 → Instance State
- Status Checks

---

2. Check Application Load Balancer (ALB)

Verify:

- Is the ALB healthy?
- Are all targets healthy?
- Is the listener configured correctly?

Example:

```
Target Group

EC2-1 ✅ Healthy
EC2-2 ❌ Unhealthy
```

If all targets are unhealthy, users cannot access the application.

---

3. Check Network Configuration

Verify:

- Security Groups
- Network ACLs
- Route Tables
- Internet Gateway
- Public IP / Elastic IP (if applicable)

Example:

SSH doesn't work?

Check:

- Port **22** allowed?
- Correct source IP?

- Correct route to the Internet Gateway?

---

4. Check Application & Logs

Verify services:

```
systemctl status nginx
```

or

```
systemctl status httpd
```

or

```
systemctl status tomcat
```

Check logs:

Ubuntu:

```
tail -100 /var/log/syslog
```

Amazon Linux/RHEL:

```
tail -100 /var/log/messages
```

Application logs:

```
tail -100 /var/log/nginx/error.log
```

---

5. Check Monitoring & Alerts

Check:

- CloudWatch Metrics
- CloudWatch Alarms
- CloudTrail Logs

Example:

CloudWatch shows:

```
CPU = 98%
```

Possible cause:

- Server overloaded
- Auto Scaling didn't trigger

CloudTrail may show:

- Security Group modified
- EC2 stopped
- Route Table changed

Real-Time Troubleshooting Flow

```
User
   |
Website Not Working
   |
1. EC2 Running?
   |
2. ALB Healthy?
   |
3. Network OK?
   |
4. Application Running?
   |
5. CloudWatch & CloudTrail
```

Common Interview Questions

Q1: If SSH is not working, what do you check?

Answer:

- Security Group
- Network ACL
- Route Table
- Internet Gateway
- Public IP
- SSH service

Q2: If the ALB shows unhealthy targets, what do you check?

Answer:

- Application service is running.
- Health check path is correct.

- Security Group allows traffic from the ALB.
- Application is listening on the correct port.

Q3: If CPU is 100%, what do you do?

Answer:

- Run `top` to identify the process.
- Check application logs.
- Restart the service if appropriate.
- Scale out using Auto Scaling if needed.

Easy Memory Trick

Remember **EANAM**:

- **E** → EC2 Instance
- **A** → ALB
- **N** → Network (Security Groups, NACLs, Routes)
- **A** → Application & Logs
- **M** → Monitoring (CloudWatch/CloudTrail)

2-Minute Interview Summary

"When a production website is down, I first verify that the EC2 instance is healthy and running. Next, I check the Application Load Balancer and target group health. Then I verify the network configuration, including Security Groups, Network ACLs, Route Tables, and the Internet Gateway. After that, I confirm that the web server and application services are running and review their logs for errors. Finally, I check CloudWatch metrics and alarms, along with CloudTrail logs, to identify any resource issues or recent configuration changes. This systematic approach helps isolate the root cause quickly while minimizing downtime."

☆ Interview Tip (Production Mindset)

Interviewers like candidates who emphasize **service restoration first**, then **root cause analysis**.

A strong closing statement is:

"In a production environment, my priority is to restore the service as quickly and safely as possible. Once the website is back online, I perform a root cause analysis, document the incident, and implement preventive measures such as monitoring, alerts, or configuration improvements to reduce the chance of recurrence."